R Programming Notes for Absolute Beginners

Part 4: Control Statements, Loops, Iteration and Functions

How to make R decide, repeat and reuse. With flowcharts, step-by-step diagrams, real-life examples solved line by line, the dangers of infinite loops, common mistakes, and practice questions.

Contents

(If the contents list looks empty, right-click it in Word and choose Update Field.)

1. Giving Your Program a Brain

So far your code has run in a straight line, top to bottom, like reading a shopping list. Real programs need more. They must make choices ("if it is raining, take an umbrella"), repeat jobs ("wash every plate"), and package steps so they can be reused ("the recipe for tea"). R gives you three sets of tools for this:

Tool	The question it answers	Everyday example
Control statements: if, else, switch	Which path should the program take?	If the light is red, stop; otherwise go
Loops: for, while, repeat	How do I do this many times?	For each student, print the result
Functions	How do I save these steps under a name and reuse them?	A recipe called make_tea()
Iteration tools: sapply, map, across	How do I apply a function to many things neatly?	Calculate the average for every column

> **In plain words:** Control statements and loops are about the ORDER and REPETITION of steps. Functions are about PACKAGING steps. The magic happens when you combine all three, and by the end of these notes you will.

1.1 Conditions: the questions R can answer

Every decision starts with a condition: something that is either TRUE or FALSE. You met these in the operators notes. Here is a quick refresher, because everything in this part depends on it.

Code (type this)	What it does, in simple words	What you will see
`5 > 3`	A comparison. Is 5 bigger than 3? The answer is TRUE or FALSE.	`[1] TRUE`
`5 == 5`	Double equals asks "are these equal?". (A single = would assign a value instead.)	`[1] TRUE`
`"a" == "A"`	Text is case sensitive, so a small a and a capital A are different.	`[1] FALSE`
`age <- 20` `age >= 18 & age < 65`	& means AND: both sides must be TRUE. Here the age is at least 18 AND below 65.	`[1] TRUE`
`age < 18 | age > 65`	| means OR: at least one side must be TRUE.	`[1] FALSE`
`!(age > 18)`	! means NOT: it flips TRUE to FALSE.	`[1] FALSE`
`TRUE && FALSE`	&& is a "single answer" AND. Use && and || inside if statements, because they expect ONE TRUE/FALSE.	`[1] FALSE`
`c(TRUE, FALSE) & c(TRUE, TRUE)`	The single & works on whole vectors, item by item. It answers many questions at once.	`[1]  TRUE FALSE`
`"b" %in% c("a", "b", "c")`	%in% asks "is this value somewhere in that list?".	`[1] TRUE`

2. Decision Making: if, else and switch

Decision making lets your program choose a path. Think of a fork in the road: the condition is the signpost, and R takes the path that matches. R has five decision tools:

- **if:** do something only when a condition is TRUE.
- **if ... else:** choose between two paths.
- **if ... else if ... else (the ladder):** choose between several paths.
- **nested if:** a decision inside another decision.
- **switch:** pick one of many named options.

A flowchart is a map of a program. Boxes are steps, and diamonds are questions with a Yes road and a No road. We draw one for each tool so you can see the path R follows.

2.1 The if statement

The if statement asks one question. If the answer is TRUE, R runs the code inside the curly brackets { }. If FALSE, R skips the whole block and carries on after it.

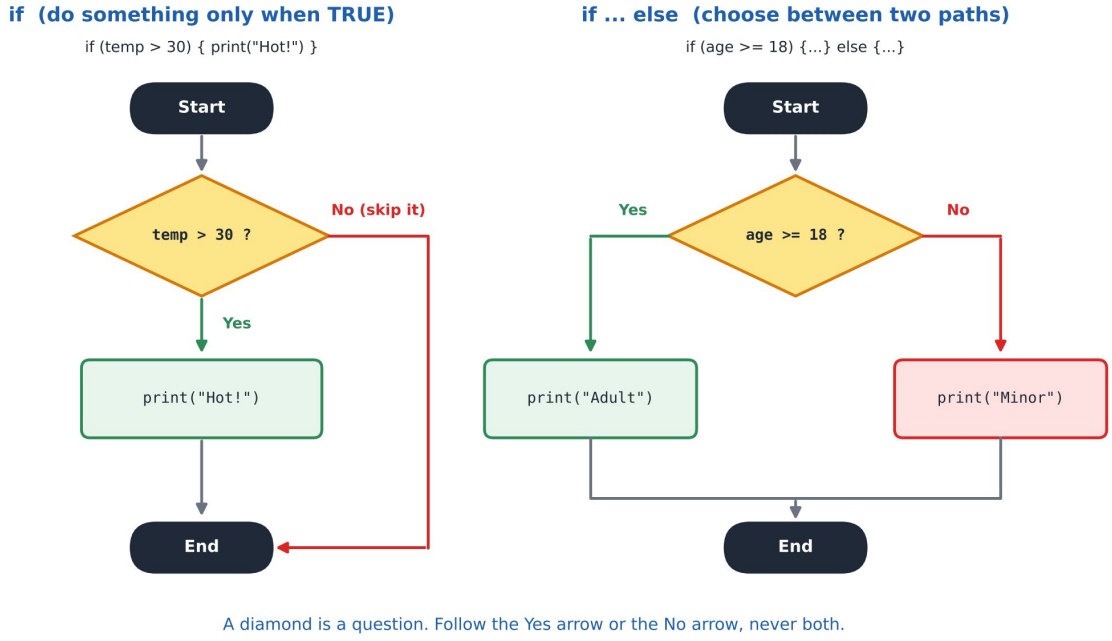


Figure 1: Flowcharts for if (left) and if ... else (right).

Code (type this)	What it does, in simple words	What you will see
`a <- 76` `b <- 67`	Two numbers to compare.	*(nothing is shown)*

`if (a > b) {`
`  difference <- a - b`
`  print("condition a > b is TRUE")`
`  print(paste("Difference between a and b is:", difference))`
`}`
	R checks a > b. It is TRUE (76 is bigger than 67), so R runs everything inside the curly brackets: it calculates the difference, then prints two lines. paste() glues the words and the number together.	

`[1] "condition a > b is TRUE"` `[1] "Difference between a and b is: 9"`		

Code (type this)	What it does, in simple words	What you will see

`if (a < b) {`
`  print("a is smaller than b")`
`}`
	This time the condition is FALSE, so R skips the block. Nothing at all happens.	*(nothing is shown)*

`temp <- 32`
`if (temp > 30) {`
`  print("It is hot. Drink water!")`
`}`
	A real-life example: if the temperature is above 30, show a reminder.	

`[1] "It is hot. Drink water!"`		

> **Tip:** The condition goes in round brackets ( ). The code to run goes in curly brackets { }. Indenting the code inside (two spaces) makes it easy to read.

2.2 if ... else

Sometimes you need a plan B. if ... else gives two roads: one when the condition is TRUE, and another when it is FALSE. Exactly one of them runs, never both.

Code (type this)	What it does, in simple words	What you will see

`a <- 67`
`b <- 76`
`if (a > b) {`
`  difference <- a - b`
`  print("condition a > b is TRUE")`
`  print(paste("Difference between a and b is:", difference))`
`} else {`
`  difference <- a - b`
`  print("condition a > b is FALSE")`
`  print(paste("Difference between a and b is:", difference))`
`}`
	Now a is 67 and b is 76, so a > b is FALSE. R skips the first block and runs the block after else.	

`[1] "condition a > b is FALSE"` `[1] "Difference between a and b is: -9"`		

`age <- 17`
`if (age >= 18) {`
`  print("You can vote")`
`} else {`
`  print("Too young to vote")`
`}`
	A real-life check: 17 is not 18 or more, so the else path runs.	

`[1] "Too young to vote"`		
`day <- "Sat"` `if (day == "Sat" || day == "Sun") print("Weekend") else print("Weekday")`	For very short choices you can write everything on one line, without curly brackets. || means OR: Saturday or Sunday.	`[1] "Weekend"`

> **Watch out:** When you use curly brackets, write } else { on the SAME line. If else starts a new line at the top level of a script, R thinks the if statement already ended and shows an error (see section 2.7).

2.3 The if ... else if ... else ladder

When there are more than two possibilities, chain the conditions into a ladder. R checks them from the top, in order. The first condition that is TRUE wins, R runs that block, and all the rest are skipped. The final else catches everything that did not match.

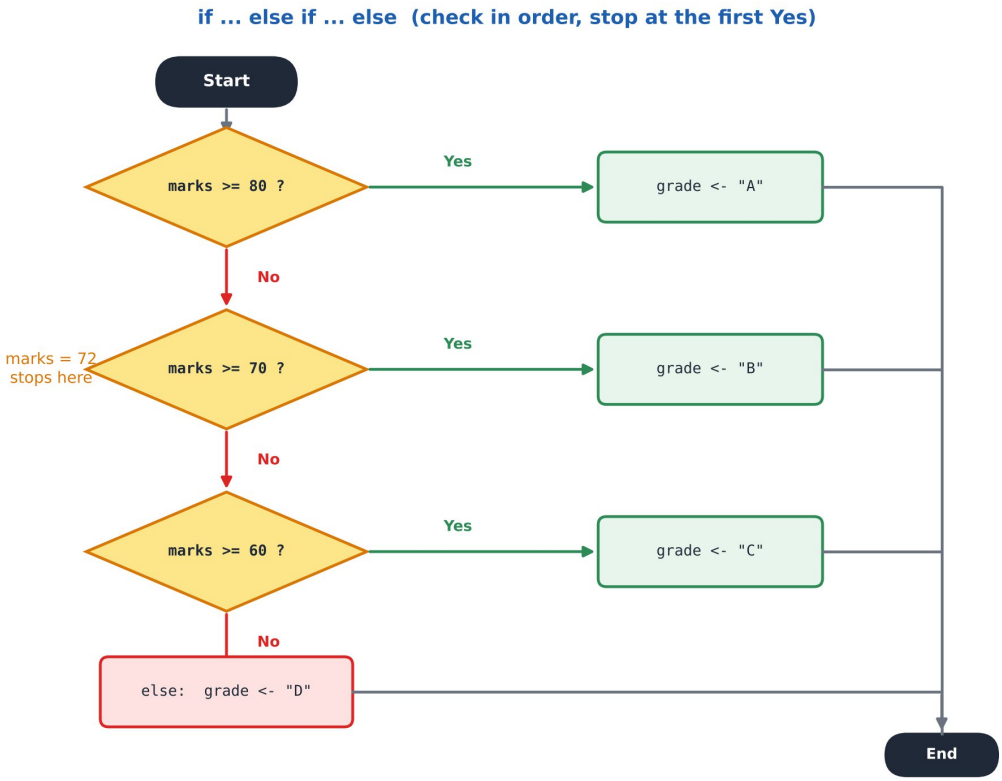


Figure 2: The ladder. R walks down until it finds the first Yes, then jumps to the end.

Code (type this)	What it does, in simple words	What you will see

`marks <- 72`
`if (marks >= 80) {`
`  grade <- "A"`
`} else if (marks >= 70) {`
`  grade <- "B"`
`} else if (marks >= 60) {`
`  grade <- "C"`
`} else {`
`  grade <- "D"`
`}`
`grade`
	R tests 72 >= 80 (No), then 72 >= 70 (Yes!), so grade becomes "B". It never even looks at the last two conditions. The last line shows the grade.	`[1] "B"`

`age <- 8`
`if (age < 5) {`
`  price <- 0`
`} else if (age < 13) {`
`  price <- 50`
`} else if (age < 60) {`
`  price <- 100`
`} else {`
`  price <- 40`
`}`
`paste("Ticket price:", price)`
	A real-life ticket price: free under 5, 50 for children under 13, 100 for adults under 60, and 40 for seniors. An 8-year-old pays 50.	`[1] "Ticket price: 50"`

`a <- 67; b <- 76; d <- 99`
`if (a > b && b > d) {`
`  print("a > b > d")`
	Here each condition joins two comparisons with && (AND). Only the	`[1] "a < b < d"`

Code (type this)	What it does, in simple words	What you will see

`} else if (a < b && b > d) {`
`  print("a < b > d")`
`} else if (a < b && b < d) {`
`  print("a < b < d")`
`}`
	third is TRUE. Note there is no final else, so if none matched, nothing would print.	

In plain words: Order matters! If you wrote marks >= 60 first, a score of 95 would stop at the first condition and wrongly become a C. Always put the most specific (strictest) condition first.

2.4 Nested if-else

A nested if is a decision inside a decision, like a security gate: first "do you have an account?", and only if yes, "is the password right?". The inner question is asked only when the outer one is TRUE.

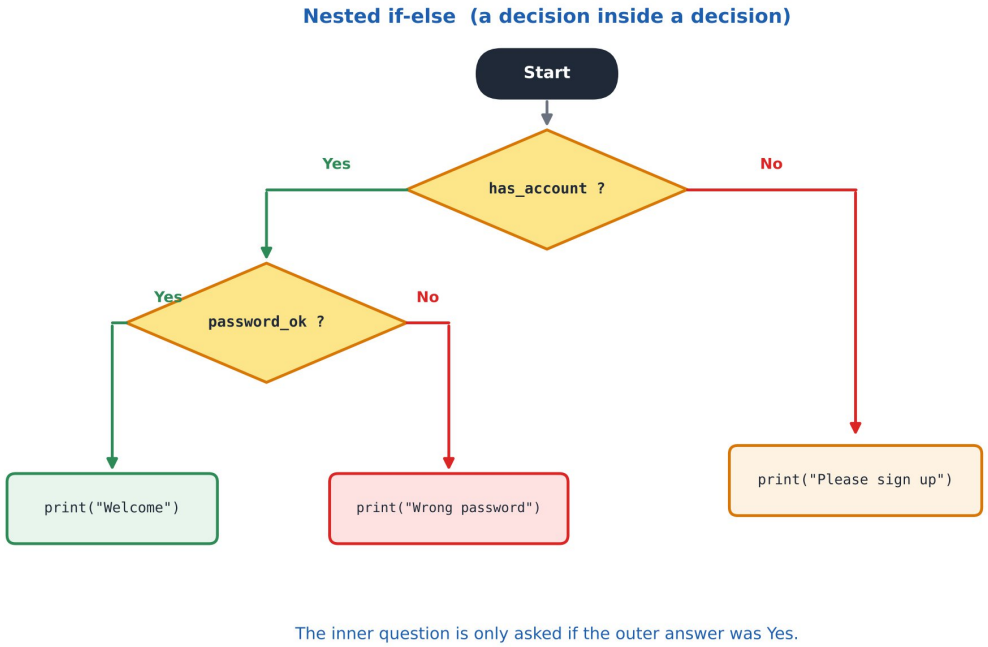


Figure 3: A nested decision. The inner diamond is reached only after a Yes at the outer one.

Code (type this)	What it does, in simple words	What you will see

`has_account <- TRUE`
`password_ok <- FALSE`
`if (has_account) {`
`  if (password_ok) {`
`  print("Welcome")`
`  } else {`
`  print("Wrong password")`
`  }`
`} else {`
`  print("Please sign up")`
`}`
	has_account is TRUE, so R enters the first block. Inside it, password_ok is FALSE, so the inner else runs.	`[1] "Wrong password"`

`a <- 10`
`b <- 11`
`if (a == 10) {`
`  if (b == 10) {`
`  print("a:10 b:10")`
`  } else {`
	Step by step: a equals 10 (yes), so go inside; b equals 10 (no), so print the inner else.	`[1] "a:10 b:11"`

Code (type this)	What it does, in simple words	What you will see

`    print("a:10 b:11")`
`  }`
`} else {`
`  if (a == 11) {`
`    print("a:11 b:10")`
`  } else {`
`    print("a:11 b:11")`
`  }`
`}`		

`balance <- 5000`
`amount <- 8000`
`if (amount > 0) {`
`  if (amount <= balance) {`
`    print("Withdrawal approved")`
`  } else {`
`    print("Insufficient funds")`
`  }`
`} else {`
`  print("Enter a positive amount")`
`}`
	A cash machine: first check the amount makes sense, then check there is enough money.	

`[1] "Insufficient funds"`		

> **Tip:** If your nesting goes more than two or three levels deep, it becomes hard to read. Try joining conditions with && or use an else if ladder instead.

2.5 switch: pick from a menu

When one value decides between many fixed options, switch is tidier than a long ladder. You give it the value, then a list of cases. R jumps straight to the matching case. You can match by position (1, 2, 3) or by name.

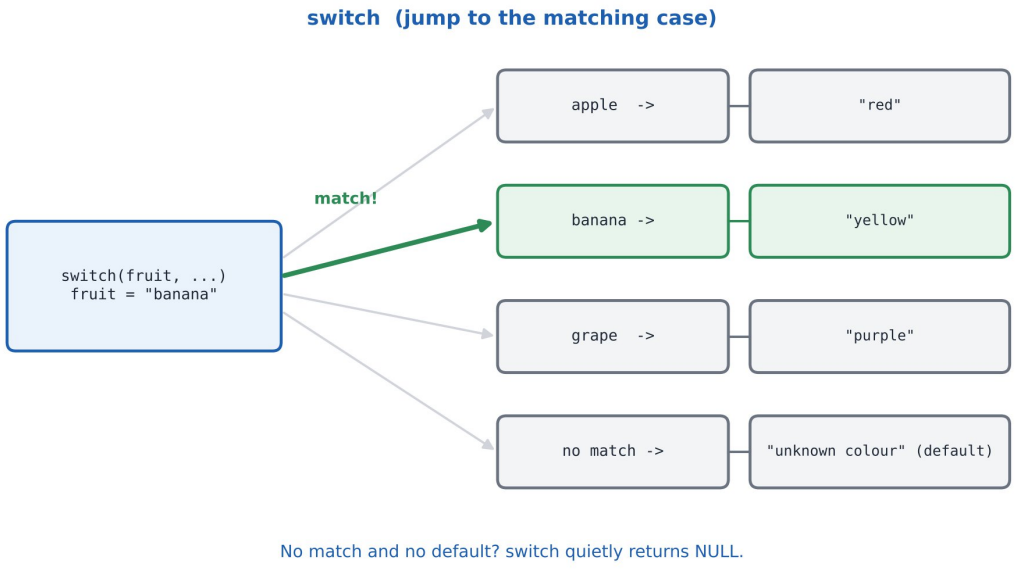


Figure 4: switch looks for a matching case and returns its result.

Code (type this)	What it does, in simple words	What you will see
`switch(2, "red", "green", "blue")`	By position: the first value is 2, so R picks the 2nd item.	`[1] "green"`

Code (type this)	What it does, in simple words	What you will see
`switch("banana", apple = "red", banana = "yellow", grape = "purple")`	By name: R finds the case called banana and gives back its value.	`[1] "yellow"`
`switch("kiwi", apple = "red", banana = "yellow", "unknown colour")`	An unnamed last item acts as a default: it is used when nothing matches.	`[1] "unknown colour"`
`z <- switch("kiwi", apple = "red", banana = "yellow")` `is.null(z)`	With no match and no default, switch quietly returns NULL (nothing).	`[1] TRUE`
`switch("b", a = , b = "Group AB", c = "Group C")`	An empty case (a = ,) "falls through" to the next one. So both "a" and "b" give "Group AB". Handy when several inputs share an answer.	`[1] "Group AB"`
`day_num <- 3` `switch(day_num, "Mon", "Tue", "Wed", "Thu")`	Real-life example: turn a day number into a day name.	`[1] "Wed"`

`light <- "red"`
`action <- switch(light, red = "Stop", yellow = "Get ready", green = "Go", "Light not working")`
`action`
	Traffic light: choose the action by the colour, with a default message.	`[1] "Stop"`

2.6 ifelse(): deciding for a whole vector at once

A normal if statement looks at ONE TRUE or FALSE. But R loves vectors, so it has ifelse() for deciding on many values at once. It is a favourite for creating new columns.

Code (type this)	What it does, in simple words	What you will see
`scores <- c(45, 72, 58, 90)`	Four exam scores.	*(nothing is shown)*
`ifelse(scores >= 50, "Pass", "Fail")`	For each score, give "Pass" if the test is TRUE, otherwise "Fail". One line replaces four if statements.	
`[1] "Fail" "Pass" "Pass" "Pass"`		
`ifelse(scores >= 80, "A", ifelse(scores >= 60, "B", "C"))`	Nesting ifelse() inside ifelse() makes a ladder for the whole vector.	`[1] "C" "B" "C" "A"`
`sum(scores >= 50)`	A neat counting trick: TRUE counts as 1, so this counts how many passed.	`[1] 3`

	if / else	ifelse()
Looks at	ONE condition (a single TRUE/FALSE)	A whole vector of conditions
Used for	Choosing which CODE runs	Choosing which VALUES to produce
Example	if (x > 5) print("big")	ifelse(x > 5, "big", "small")

2.7 Mistakes beginners make with decisions

Red output means an error. These lines are left commented out in the ready-made R file so the file still runs from start to finish.

Code (type this)	What it does, in simple words	What you will see
`x <- 8`	A number to test.	*(nothing is shown)*

`if (x > 5) {`
`  print("big")`
`}`
`else {`
`  print("small")`
`}`
	MISTAKE: else on a new line at the top level of a script. R thinks the if statement ended at the first closing bracket, so a lonely else makes no sense. (Inside a function's curly brackets this is allowed, but it is a bad habit.)	

`Error: unexpected 'else' in "else"`		

`if (x > 5) {`
`  print("big")`
`} else {`
`  print("small")`
`}`
	FIX: put } else { together on one line.	`[1] "big"`

`marks3 <- c(45, 72, 58)`	Three marks (a vector with more than one value).	*(nothing is shown)*
`if (marks3 >= 50) print("Pass")`	MISTAKE: if wants ONE TRUE/FALSE but got three. Since R 4.2 this is an error.	
`Error in if (marks3 >= 50) print("Pass") :` `  the condition has length > 1`		
`ifelse(marks3 >= 50, "Pass", "Fail")`	FIX: use ifelse() for a whole vector (or a loop to handle items one at a time).	
`[1] "Fail" "Pass" "Pass"`		
`if (NA > 3) print("yes")`	MISTAKE: the condition is NA (unknown), and R cannot decide a path.	
`Error in if (NA > 3) print("yes") :` `  missing value where TRUE/FALSE needed`		
`value <- NA` `if (!is.na(value) && value > 3) print("big") else print("missing or small")`	FIX: check for NA first. Because && stops at the first FALSE, the risky comparison is never reached.	`[1] "missing or small"`
`y <- 5`	A number to test.	*(nothing is shown)*
`if (y = 5) print("five")`	MISTAKE: a single = tries to assign, not compare.	
`Error: unexpected '=' in "if (y ="`		
`if (y == 5) print("five")`	FIX: use double equals == to compare.	`[1] "five"`

`if (FALSE)`
`  print("one")`
`  print("two")`
	TRAP: without curly brackets, if controls ONLY the next line. The second print always runs. Use { } whenever you have more than one line.	`[1] "two"`

`"10" > 9`	TRAP: 9 is turned into text and the two are compared alphabetically, where "10" comes before "9". Make sure numbers are numbers, not text.	`[1] FALSE`

3. Loops: Repeating Things

Imagine washing 100 plates. You would not write 100 separate instructions. You would say: "for each plate, wash it". That is a loop: a way of repeating the same steps again and again, changing something small each time.

R has three kinds of loop. Choose by asking one question: do I know how many times to repeat?

Loop	Use it when...	Real-life example
for	You know what to loop over (a list of items, or a number of times)	For each student in the class, print the result
while	You repeat until a condition stops being TRUE; you do not know how many rounds	While the balance is below the target, keep saving
repeat	You want to run at least once and stop yourself with break	Keep asking for a password until it is correct

3.1 The for loop

A for loop takes a collection of items and visits them one at a time. On each visit it puts the current item into a helper variable (usually called i), runs the code inside { }, then moves to the next item. When the items run out, the loop ends.

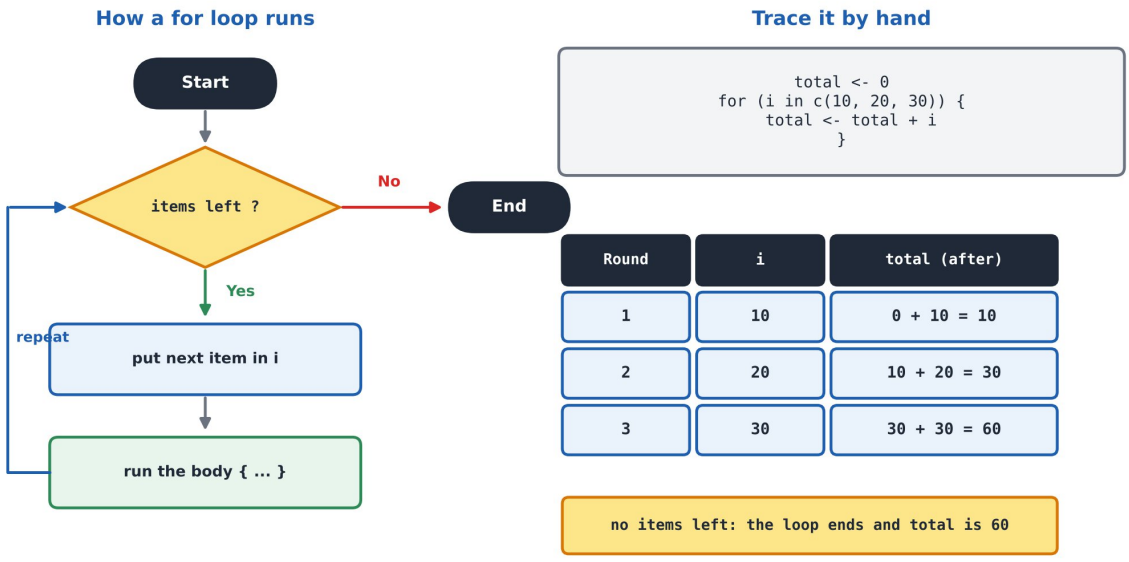


Figure 5: A for loop asks "are there items left?" over and over. The table traces total, round by round.

Code (type this)	What it does, in simple words	What you will see

`for (i in 1:5) {`
`  print(i)`
`}`
	i takes the values 1, 2, 3, 4, 5 in turn. In each round, the body prints the current value of i. So print() runs five times.	`[1] 1`
		`[1] 2`
		`[1] 3`
		`[1] 4`
		`[1] 5`

Code (type this)	What it does, in simple words	What you will see

`for (fruit in c("apple", "banana", "mango")) {`
`  print(paste("I like", fruit))`
`}`
	The helper variable does not have to be i. Here it is called fruit, and it visits each piece of text in the vector.	`[1] "I like apple"`
		`[1] "I like banana"`
		`[1] "I like mango"`

`total <- 0`
`for (i in c(10, 20, 30)) {`
`  total <- total + i`
`}`
`total`
	The running-total pattern. Start total at 0. Each round adds the current item: 0+10, then 10+20, then 30+30. The trace table in the picture shows this.	`[1] 60`

`prices <- c(120, 80, 45)`
`bill <- 0`
`for (p in prices) {`
`  bill <- bill + p`
`}`
`bill`
	A real-life bill: add up the price of each item on a receipt. (Of course sum(prices) does it in one go, but this shows how the loop thinks.)	`[1] 245`

`scores <- c(70, 85, 60)`
`for (i in seq_along(scores)) {`
`  cat("Student", i, "scored", scores[i], "\n")`
`}`
	seq_along(scores) gives the positions 1, 2, 3. Looping over positions lets you use both the number i and the value scores[i]. cat() prints a tidy line and \n starts a new line.	`Student 1 scored 70`
		`Student 2 scored 85`
		`Student 3 scored 60`

`marks <- c(Ana = 80, Ben = 65)`
`for (nm in names(marks)) {`
`  cat(nm, "->", marks[nm], "\n")`
`}`
	Loop over the names of a named vector, then use each name to pick out its value.	`Ana -> 80`
		`Ben -> 65`

`students <- list(Ana = c(70, 80), Ben = c(60, 65))`
`for (nm in names(students)) {`
`  cat(nm, "average:", mean(students[[nm]]), "\n")`
`}`
	Looping over a list. students[[nm]] opens the compartment with that name, and mean() averages the scores inside.	`Ana average: 75`
		`Ben average: 62.5`

`for (i in 1:5) {`
`  cat("7 x", i, "=", 7 * i, "\n")`
`}`
	A multiplication table for 7: a classic first loop.	`7 x 1 = 7`
		`7 x 2 = 14`
		`7 x 3 = 21`
		`7 x 4 = 28`
		`7 x 5 = 35`

`squares <- numeric(5)`
`for (i in 1:5) {`
`  squares[i] <- i^2`
`}`
`squares`
	Saving results from a loop. First make an empty vector of the right size (numeric(5) is five zeros), then fill in one box per round. Preparing the space first is faster than growing a vector.	`[1]  1  4  9 16 25`

`evens <- c()`
`for (i in 1:10) {`
`  if (i %% 2 == 0) {`
`  evens <- c(evens, i)`
`  }`
`}`
`evens`
	A loop with a decision inside (combining two tools!). %% gives the remainder, so i %% 2 == 0 means i is even. Each even number is added to the vector.	`[1]  2  4  6  8 10`

Nested loops: a loop inside a loop

A nested loop is like a clock: for every one move of the hour hand, the minute hand goes all the way round. The inner loop completes fully for each step of the outer loop.

Code (type this)	What it does, in simple words	What you will see

`for (i in 1:3) {`
`  for (j in 1:3) {`
`  cat(i * j, " ")`
`  }`
`  cat("\n")`
`}`
	The outer loop picks a row (i). For each row, the inner loop visits every column (j) and prints i times j on the same line. After the inner loop finishes, cat("\n") starts a new line. The result is a small multiplication grid.	`1 2 3`
		`2 4 6`
		`3 6 9`

Two famous for-loop traps

Code (type this)	What it does, in simple words	What you will see
`for (i in 1:3) i`	TRAP: inside a loop, R does not show values automatically. You must use print() or cat(). This loop runs three times but you see nothing.	*(nothing is shown)*

`for (i in 1:3) print(i)`	FIX: add print().	`[1] 1`
		`[1] 2`
		`[1] 3`

`x <- c()` `for (i in 1:length(x)) print(i)`	TRAP: x is empty, so length(x) is 0, and 1:0 counts DOWN as 1, 0. The loop runs twice when it should not run at all.	`[1] 1` `[1] 0`
`for (i in seq_along(x)) print(i)`	FIX: seq_along() gives an empty sequence for an empty vector, so the loop correctly does nothing. Prefer seq_along(x) over 1:length(x).	*(nothing is shown)*

3.2 The while loop

A while loop checks a condition BEFORE every round. As long as the condition is TRUE, it runs the body. When the condition becomes FALSE, it stops. Because the check comes first, a while loop can run zero times.

There is one golden rule: something inside the body must eventually make the condition FALSE. Otherwise the loop never ends (see section 3.5).

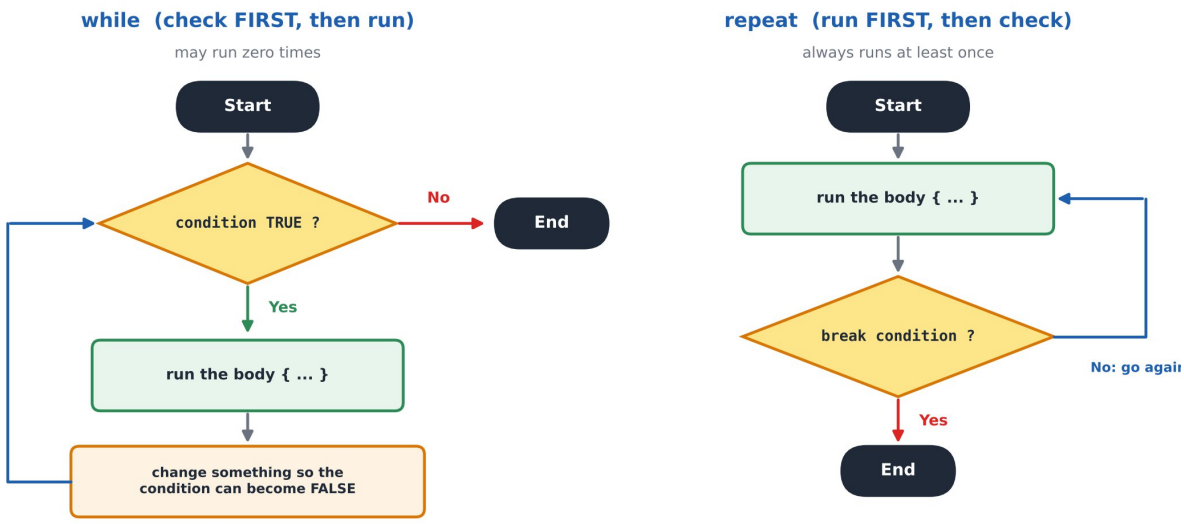


Figure 6: while checks first (left). repeat runs first and checks afterwards (right).

Code (type this)	What it does, in simple words	What you will see

`count <- 1`
`while (count <= 3) {`
`  print(count)`
`  count <- count + 1`
`}`
	Start count at 1. While count is 3 or less, print it and add 1. After the third round count is 4, the condition is FALSE, and the loop ends. The line count <- count + 1 is what makes the loop able to finish.	`[1] 1`
		`[1] 2`
		`[1] 3`

`n <- 3`
`while (n > 0) {`
`  cat(n, "...\n")`
`  n <- n - 1`
`}`
`cat("Liftoff!\n")`
	A rocket countdown. n goes down by 1 each round, until it hits 0.	`3 ...`
		`2 ...`
		`1 ...`
		`Liftoff!`

`balance <- 1000`
`years <- 0`
`while (balance < 2000) {`
`  balance <- balance * 1.10`
`  years <- years + 1`
`}`
`years`
	Real-life savings: money grows 10% a year. How many years until 1000 doubles? We do not know the number of rounds in advance, which is exactly when while is the right tool.	`[1] 8`

`round(balance, 2)`	The balance after those 8 years.	`[1] 2143.59`

`n <- 51`
`while (n %% 7 != 0) {`
`  n <- n + 1`
`}`
`n`
	Find the first number from 51 upwards that 7 divides exactly. != means "not equal". It stops at 56.	`[1] 56`

3.3 The repeat loop and break

repeat runs forever on its own, so you must stop it yourself with break. It always runs at least once, which makes it perfect for "keep trying until it works".

Code (type this)	What it does, in simple words	What you will see

`i <- 0`
`repeat {`
`  i <- i + 1`
`  if (i == 4) {`
`  break`
`  }`
`  print(i)`
`}`
	Each round adds 1 to i. When i reaches 4, the if statement runs break, which leaves the loop immediately (before print runs for 4). So only 1, 2, 3 are printed.	`[1] 1`
		`[1] 2`
		`[1] 3`

`i`	After the loop, i is still 4, because that is where it stopped.	`[1] 4`

3.4 break and next: steering a loop

Two special words let you steer a loop from the inside:

- **next:** skip the rest of THIS round and jump to the next round. The loop carries on.
- **break:** stop the whole loop right now and jump to the first line after the loop. Any rounds that were left never happen.

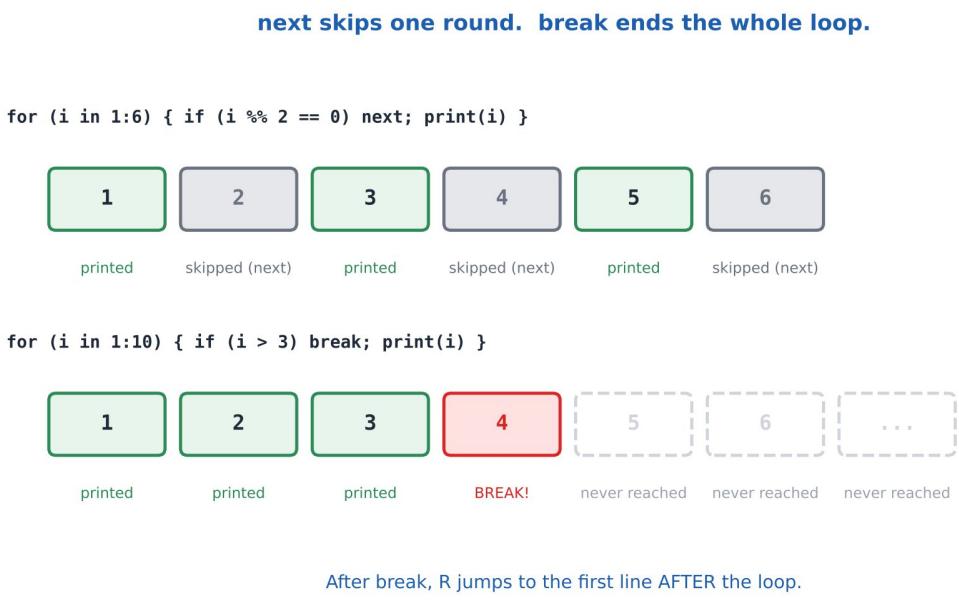


Figure 7: next skips one box; break ends the loop and the remaining boxes are never visited.

Code (type this)	What it does, in simple words	What you will see

`for (i in 1:6) {`
`  if (i %% 2 == 0) {`
`  next`
`  }`
`  print(i)`
`}`
	When i is even, next skips the print and the loop moves on to the next i. So only the odd numbers are printed.	`[1] 1`
		`[1] 3`
		`[1] 5`

`for (i in 1:10) {`
`  if (i > 3) {`
`  break`
`  }`
`  print(i)`
`}`
	When i becomes 4, break ends the loop. The numbers 5 to 10 are never visited, so only 1, 2, 3 appear.	`[1] 1`
		`[1] 2`
		`[1] 3`

`numbers <- c(4, 9, -2, 7)`	Real-life data check: stop as soon as an impossible (negative) value	

Code (type this)	What it does, in simple words	What you will see

`for (x in numbers) {`
`  if (x < 0) {`
`  cat("Found a negative number:", x, "- stopping\n")`
`  break`
`  }`
`  cat("OK:", x, "\n")`
`}`
	is found. 4 and 9 pass, -2 triggers the break, and 7 is never checked.	

`OK: 4`
`OK: 9`
`Found a negative number: -2 - stopping`		

`total <- 0`
`for (price in c(50, NA, 30)) {`
`  if (is.na(price)) {`
`  next`
`  }`
`  total <- total + price`
`}`
`total`
	Skip missing prices with next, and add up the rest.	`[1] 80`

> **In plain words:** What happens in a nested loop? break and next only affect the loop they are written in (the innermost one). The outer loop carries on as normal.

3.5 Infinite loops: when a loop never ends

An infinite loop is a loop whose condition never becomes FALSE and which never meets a break. It is one of the most common beginner accidents, and it is good to know exactly what it does to your computer so you are not frightened when it happens.

What actually happens

- **R never finishes.** The loop spins forever, so the next lines of your script never run. RStudio stays "busy" and the Console stops accepting new commands.
- **The processor works non-stop.** R keeps one CPU core at about 100%. Your laptop fan may spin up and the battery drains faster.
- **Memory can climb.** If the loop keeps growing something (for example x <- c(x, 1) adds one more item every round), R must copy and enlarge the vector again and again. Memory (RAM) keeps filling up. Eventually R may stop with the error "cannot allocate vector of size ...", your computer may become very slow as it starts using the disk as extra memory, or R may crash and you lose any unsaved objects.
- **The Console floods.** If the loop also prints, thousands of lines scroll past and RStudio becomes sluggish.

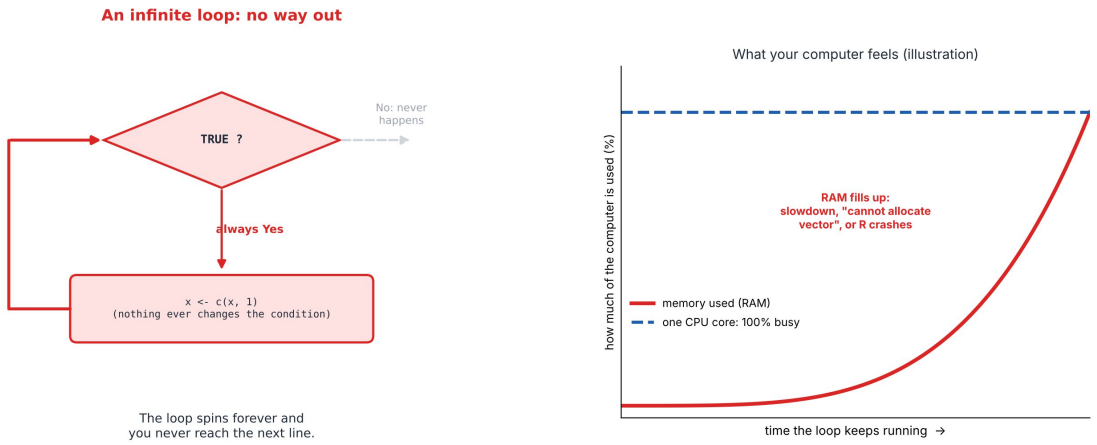


Figure 8: An infinite loop has no exit. The right-hand chart is an illustration of how a growing loop uses up memory over time; real numbers depend on your machine.

How loops become infinite (do NOT run these)

Kind	Example (never run it!)	Why it never stops

No exit at all	while (TRUE) {
	  print("hi")
	}
		TRUE is always TRUE and there is no break

Forgot to update	i <- 1
	while (i <= 5) {
	  print(i)
	}
		i never changes, so i <= 5 stays TRUE

Wrong direction	n <- 10
	while (n > 0) {
	  n <- n + 1
	}
		n grows instead of shrinking, so it never reaches 0

repeat without break	repeat {
	  x <- 1
	}
		repeat only stops when you tell it to

Decimal trap	x <- 0
	while (x != 1) {
	  x <- x + 0.1
	}
		Adding 0.1 ten times gives 0.9999999999999999, not exactly 1, so x != 1 is always TRUE

Growing forever	x <- c()
	while (TRUE) {
	  x <- c(x, 1)
	}
		Never stops AND eats more memory every round

Danger zone: Please do not type these into R to try them. They are shown only so you can recognise them. If you want to see a loop behave safely, use the guarded versions below.

Safe habits that prevent infinite loops

Code (type this)	What it does, in simple words	What you will see
`i <- 0`	A guard counter. The loop looks	`Safety stop!`

Code (type this)	What it does, in simple words	What you will see

`while (TRUE) {`
`  i <- i + 1`
`  if (i >= 1000) {`
`  cat("Safety stop!\n")`
`  break`
`  }`
`}`
`i`
	endless (while TRUE) but counts its rounds and breaks at 1000 no matter what. Adding a safety limit is a great habit whenever you are unsure.	`[1] 1000`

`x <- 0`
`while (x < 1) {`
`  x <- x + 0.1`
`}`
`x > 0.99`
	FIX for the decimal trap: use < (less than) instead of != (not equal). x climbs past 0.99 and the loop ends.	`[1] TRUE`

`attempts <- 0`
`max_attempts <- 5`
`while (attempts < max_attempts)`
`{`
`  attempts <- attempts + 1`
`}`
`attempts`
	Loop with a built-in maximum. The condition itself includes the limit, so it cannot run forever.	`[1] 5`

What to do if a loop gets stuck

- **Step 1: press Esc** with the cursor in the RStudio Console (or click the red STOP sign at the top of the Console). This interrupts R and brings the > prompt back. In a plain terminal use Ctrl + C.
- **Step 2: if RStudio is frozen,** choose Session, then Interrupt R. If that does not work, choose Session, then Terminate R. You will lose the objects in memory, but not your saved scripts.
- **Step 3: if nothing responds,** close RStudio from Task Manager (Windows) or Activity Monitor (Mac).
- **Step 4: fix the loop** (update the condition, or add a break or a counter), then run it again on a small test first.

> **Tip:** Save your script (Ctrl + S) BEFORE running any loop you are unsure about. If R has to be force-closed, your code will still be safe.

- **Prevention checklist:** 1) change something in the body that moves the condition toward FALSE; 2) add a counter limit; 3) test on a small number first, such as 3 rounds; 4) prefer for loops when you know the number of rounds; 5) avoid printing thousands of lines.

3.6 Loops versus vectorised code

R is built to work on whole vectors at once. For simple maths you often do not need a loop at all, and the shorter code is also much faster on big data.

Code (type this)	What it does, in simple words	What you will see
`x <- 1:5` `x * 2`	Vectorised: one line doubles every item.	`[1]  2  4  6  8 10`

`doubled <- numeric(5)`
`for (i in 1:5) {`
`  doubled[i] <- x[i] * 2`
`}`
`doubled`
	The loop version gives the same answer, but takes more lines.	`[1]  2  4  6  8 10`

Use a loop when...	Prefer vectorised code or apply/map when...
Each round depends on the previous one (running balance, game state)	You do the same simple maths on every item (x * 2, sum(x), mean(x))
You need to stop early with break	You are creating a new column from existing columns
Each round has a side effect, such as printing or saving a file	You want a clean one-line result
You do not know how many rounds you need (while)	Speed matters on large data

3.7 Common loop mistakes

- **Forgetting to update the while condition** (see infinite loops).
- **Using 1:length(x)** instead of seq_along(x): it misbehaves on empty vectors.
- **Forgetting print() or cat()** inside the loop.
- **Growing a vector inside a loop on big data.** Create the full-size vector first (numeric(n)) and fill it in.
- **Off-by-one errors.** 1:5 - 1 is not 1:4. Use 1:(5 - 1) or think carefully about the last round.
- **Changing the thing you are looping over** while looping over it. Make a copy if you need to change it.
- **Using = instead of ==** in the if inside a loop.

4. Iteration: Doing the Same Thing to Many Things

Iteration just means doing the same action again and again on different things, like stamping every page of a document. A loop is one way to iterate, but R has several friendlier ways, and many experienced R programmers use loops less often than you might expect.

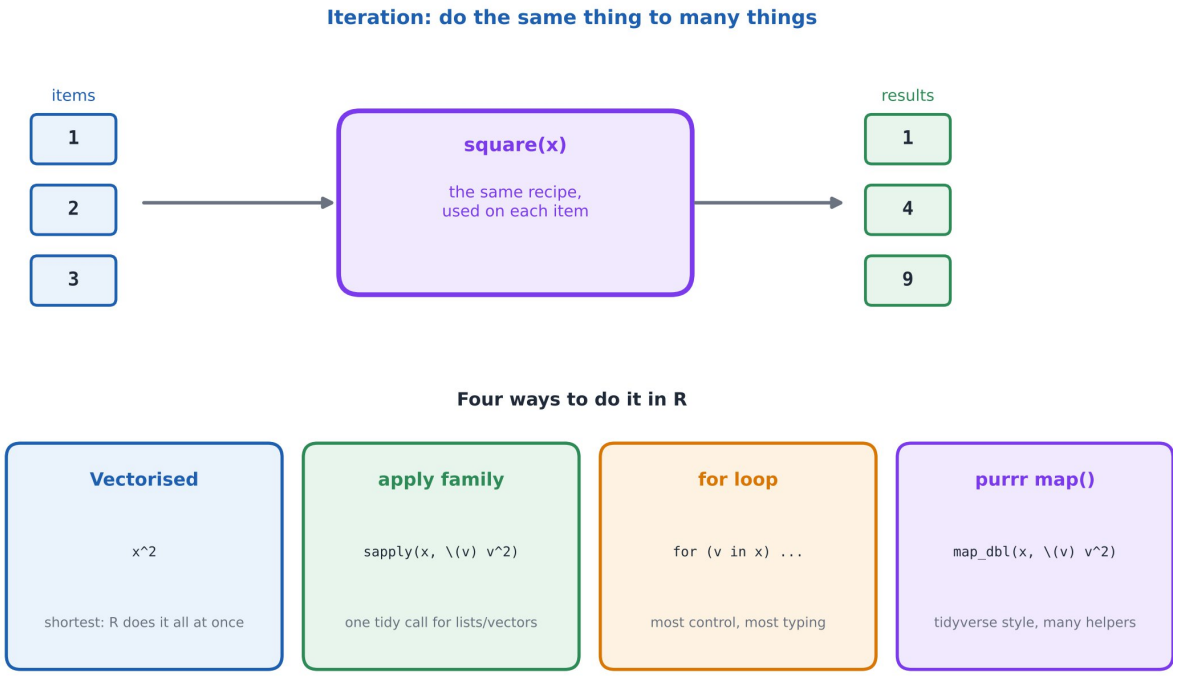


Figure 9: Iteration feeds each item through the same recipe and collects the results. R offers four ways to do it.

4.1 The apply family (base R, no extra packages)

The apply family are functions that apply another function to every item. They return the results already collected, so you do not have to build the result vector yourself. Notice that you give the FUNCTION NAME without brackets, and the apply function calls it for you.

Code (type this)	What it does, in simple words	What you will see
`sapply(1:5, function(x) x^2)`	sapply() takes each item (x) and squares it. The s stands for "simplify": the answer is tidied into a vector.	`[1]  1  4  9 16 25`
`sapply(c(a = 1, b = 2), \(x) x * 10)`	\(x) is a shorter way to write function(x) (needs R 4.1 or newer). Names are kept.	`a  b` `10 20`

`lapply(1:2, \(x) x * 2)`	lapply() always returns a LIST, one compartment per item.	`[[1]]`
		`[1] 2`
		
		`[[2]]`
		`[1] 4`

`vapply(1:3, \(x) x * 2, numeric(1))`	vapply() is the careful sibling of sapply(): you promise what type each result will be (here one number), and R checks.	`[1] 2 4 6`

Code (type this)	What it does, in simple words	What you will see
`mapply(function(x, y) x + y, 1:3, 4:6)`	mapply() goes through TWO (or more) vectors together: 1+4, 2+5, 3+6.	`[1] 5 7 9`
`m <- matrix(1:6, nrow = 2)` `apply(m, 1, sum)`	apply() works on a matrix. The 1 means "go row by row". Row 1 is 1+3+5, row 2 is 2+4+6.	`[1]  9 12`
`tapply(c(10, 20, 30, 40), c("a", "b", "a", "b"), sum)`	tapply() applies a function to groups: add up the numbers belonging to group a (10 + 30) and group b (20 + 40).	`a  b` `40 60`
`sapply(list(a = 1:3, b = 4:6), sum)`	Apply sum to each element of a list.	`a  b` `6 15`

4.2 The tidyverse way: dplyr across() and purrr map()

The tidyverse adds two friendly tools for iteration, described in the free book R for Data Science (chapter on Iteration):

- **across()** (from dplyr) does the same thing to many COLUMNS of a table at once.
- **map()** (from purrr) does something to every ITEM of a vector or list, like a more consistent sapply().

You need the packages first. Installing is done once; loading (library) is done every session. The pipe |> means "then": it passes the result on to the next step (needs R 4.1 or newer).

Code (type this)	What it does, in simple words	What you will see
`library(dplyr)` `library(tibble)`	Load the packages. dplyr gives us summarize(), mutate() and across(); tibble gives us tibble().	*(nothing is shown)*
`df <- tibble(a = c(1, 2, 3), b = c(4, 5, 6), c = c(7, 8, 9))`	A small table with three numeric columns.	*(nothing is shown)*

`df |> summarize(across(a:c, mean))`	Take df, THEN calculate the mean of every column from a to c. Without across() you would write mean(a), mean(b), mean(c) by hand. The first argument says WHICH columns; the second says WHAT to do.	`# A tibble: 1 × 3`
		`      a     b     c`
		`  <dbl> <dbl> <dbl>`
		`1     2     5     8`

`df |> summarize(n = n(), across(a:c, median))`	n() counts the rows. You can mix across() with ordinary summaries.	

`# A tibble: 1 × 4`
`      n     a     b     c`
`  <int> <dbl> <dbl> <dbl>`
`1     3     2     5     8`		

`df |> mutate(across(a:c, \(x) x * 10))`	With mutate(), across() changes every chosen column. \(x) x * 10 is a tiny throw-away function, called an anonymous function.	`# A tibble: 3 × 3`
		`      a     b     c`
		`  <dbl> <dbl> <dbl>`
		`1    10    40    70`
		`2    20    50    80`
		`3    30    60    90`

`df |> summarize(across(a:c, list(min = min, max = max)))`	A named list of functions makes several summaries per column. The new names combine the column and the function: a_min, a_max,	

Code (type this)	What it does, in simple words	What you will see
	and so on.	

`# A tibble: 1 × 6`
`  a_min a_max b_min b_max c_min c_max`
`  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>`
`1  1  3  4  6  7  9`		

`df_na <- tibble(a = c(1, NA, 3), b = c(4, 5, NA))`
`df_na |> summarize(across(a:b, \(x) mean(x, na.rm = TRUE)))`
	To pass an extra setting such as na.rm = TRUE, wrap the function in \(x) ... . This ignores missing values in each column.	`# A tibble: 1 × 2`
		`  a  b`
		`  <dbl> <dbl>`
		`1  2  4.5`

Watch out: Inside across() give the function NAME only: across(a:c, mean). Never write mean() with brackets, because then you are calling it yourself without any data, and R complains that argument "x" is missing.

Code (type this)	What it does, in simple words	What you will see
`library(purrr)`	Load purrr.	*(nothing is shown)*
`map_dbl(list(a = 1:3, b = 4:6), mean)`	map_dbl() applies mean to each item and returns numbers (dbl means decimal number). The name tells you the type of the result: map_dbl, map_chr (text), map_int, map_lgl (TRUE/FALSE).	`a b` `2 5`
`map_chr(c("a", "b"), toupper)`	Apply toupper to each piece of text and return text.	`[1] "A" "B"`

`map(1:3, \(x) x^2)`	Plain map() returns a list.	`[[1]]`
		`[1] 1`
		
		`[[2]]`
		`[1] 4`
		
		`[[3]]`
		`[1] 9`

`map2_dbl(1:3, 4:6, \(x, y) x + y)`	map2() goes through two vectors together.	`[1] 5 7 9`
`safe_log <- possibly(log, otherwise = NA)` `map_dbl(list(1, "a", 10), safe_log)`	possibly() wraps a function so that failures return NA instead of stopping everything. log("a") would normally give an error; here the bad item just becomes NA.	
`[1] 0.000000  NA 2.302585`		

`pieces <- list(y2019 = data.frame(sales = c(10, 20)),`
`  y2020 = data.frame(sales = c(30, 40)))`
`list_rbind(pieces, names_to = "year")`
	A real-world pattern: you have several small tables (one per year or file) in a list. list_rbind() stacks them into one table and records where each row came from.	`  year sales`
		`1 y2019  10`
		`2 y2019  20`
		`3 y2020  30`
		`4 y2020  40`

The same idea scales up. In R for Data Science the pattern for reading many files is: list the files with list.files(), read each with map(), then combine with list_rbind(). To save many things you use walk() or walk2(), which are like map() but throw the results away because you only care about the side

effect (such as writing a file). The code below is shown only for illustration, because it needs files on your computer.

```
paths <- list.files("data", pattern = "[.]csv$", full.names = TRUE)
all_data <- paths |>
  map(read.csv) |>
  list_rbind()
```

4.3 Which iteration tool should I use?

Tool	Best for	Example
Vectorised code	Simple maths on whole vectors or columns	x * 2
sapply / lapply / vapply	Apply a function to each item, no packages needed	sapply(x, \(v) v^2)
for loop	Steps that depend on the previous round, early stopping, side effects	running balance
while / repeat	Repeat until something becomes true	years until the savings double
across()	Same operation on many columns of a table	summarize(across(a:c, mean))
map() family	Consistent, typed iteration over lists and vectors	map_dbl(x, mean)

5. Functions: Reusable Recipes

5.1 What is a function, and why bother?

A function is a named set of steps that you can use again and again. Think of a recipe for tea: you write the steps once (boil water, add leaves, wait, pour), give the recipe a name, and from then on you just say "make tea". You have already used many functions that R provides: sum(), mean(), print(), round(). Now you will make your own.

- **Write once, use many times.** No more copying and pasting the same lines.
- **Fix once.** If there is a bug, you correct it in one place.
- **Easier to read.** area_of_circle(5) is clearer than a mystery calculation.
- **Easier to test.** You can try a small function on its own.

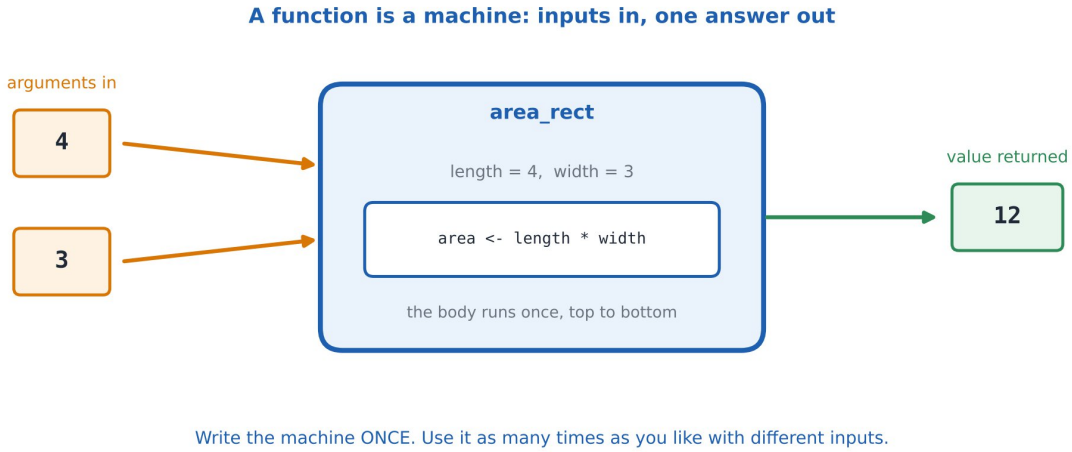


Figure 10: A function is a machine. Arguments go in, the body does the work, a value comes out.

> **In plain words:** Functions are only executed when you CALL them. Writing the function just stores the recipe; nothing happens until you use it.

5.2 The parts of a function

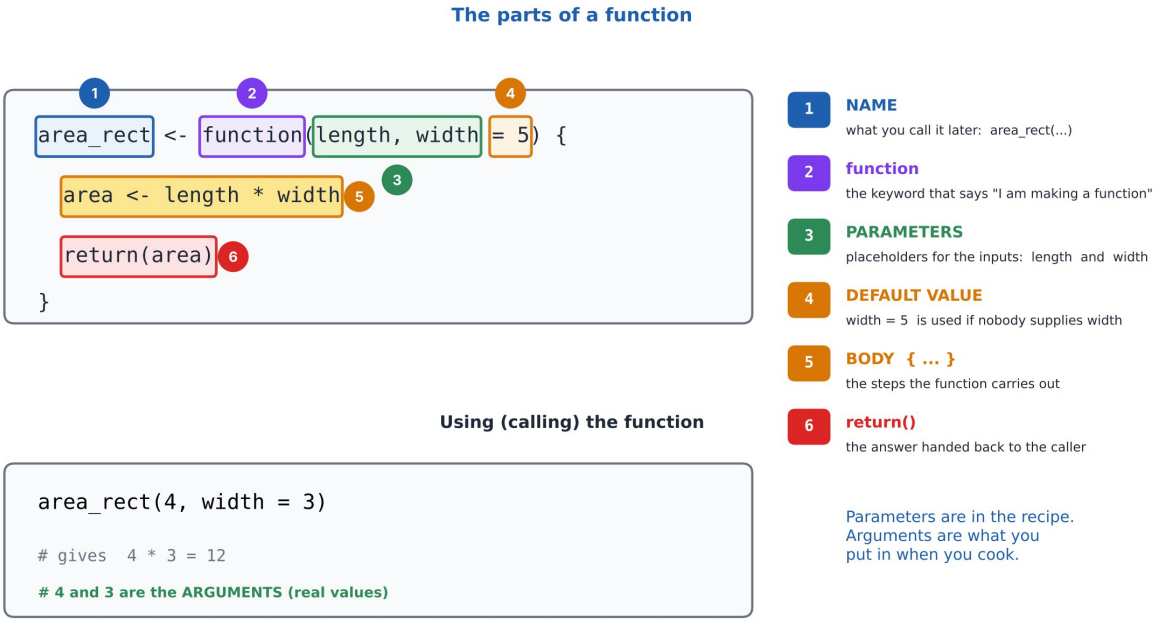


Figure 11: The six parts of a function, and how to call it.

Part	In our example	What it does
1. Name	area_rect	The label you use to call the function later. Pick a name that says what it does.
2. function keyword	function	Tells R "I am creating a function".
3. Parameters	length, width	Placeholders (empty slots) for the inputs the function will receive.
4. Default value	width = 5	A backup value, used when the caller does not supply that input.
5. Body	{ area <- length * width ... }	The steps, written between curly brackets, that run when the function is called.
6. return()	return(area)	Sends the answer back to whoever called the function.

5.3 Parameters versus arguments

These two words are often mixed up, even by experienced programmers, but the difference is simple:

- **Parameters** are the names in the function DEFINITION: the empty slots, such as length and width. Think of them as labelled seats at a table.
- **Arguments** are the real values you supply when you CALL the function, such as 4 and 3. Think of them as the guests who sit in those seats.

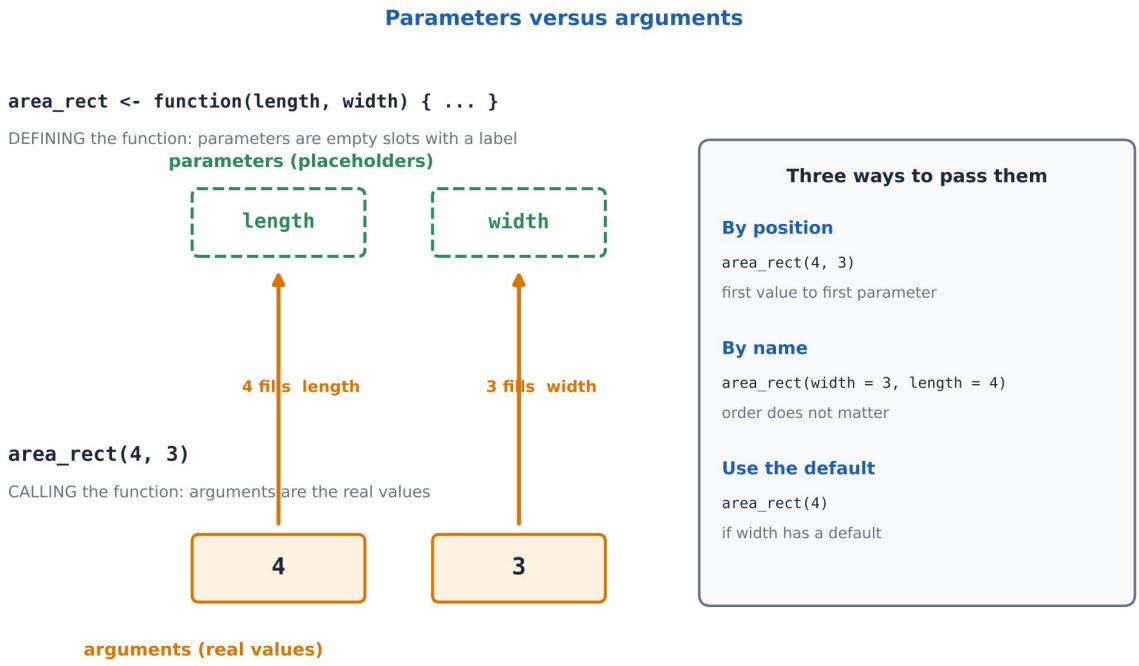


Figure 12: Parameters are placeholders in the recipe; arguments are the real values you give when you use it.

5.4 Creating and calling your first functions

Code (type this)	What it does, in simple words	What you will see

`add_num <- function(a, b) {`
`  sum_result <- a + b`
`  return(sum_result)`
`}`
	Create a function called add_num. It has two parameters, a and b. The body adds them and stores the answer, and return() sends the answer back. Nothing is shown yet: we have only stored the recipe.	*(nothing is shown)*

`total <- add_num(35, 67)`	Call the function. 35 and 67 are the ARGUMENTS: 35 goes into a, 67 goes into b. The returned value is saved in total.	*(nothing is shown)*
`total`	Look at the answer.	`[1] 102`
`add_num(1, 2)`	Call it again with different arguments. This is the whole point: one recipe, many uses.	`[1] 3`
`add_two <- function(a, b) a + b` `add_two(10, 5)`	For a one-line body you can drop the curly brackets. R also automatically returns the value of the LAST line, so return() is optional.	`[1] 15`

`calculate_square <- function(x)`
`{`
`  result <- x^2`
`  return(result)`
`}`
`calculate_square(5)`
	A function with one parameter. 5 squared is 25.	`[1] 25`

`calculate_square(-2.5)`	It works for any number you give it.	`[1] 6.25`

`area_of_circle <-`
`function(radius) {`
`  pi * radius^2`
	Single input, single output: the area of a circle is pi times radius squared. Here the last line is returned	`[1] 12.56637`

Code (type this)	What it does, in simple words	What you will see
`}` `area_of_circle(2)`	automatically.	
`f <- function(x) x^2 * 4 + x / 3` `f(4)`	An inline function: a quick one-line formula you can call with different values.	`[1] 65.33333`
`f(-2)`	The same function with a negative input.	`[1] 15.33333`
`f(0)`	And with zero.	`[1] 0`

> **Tip:** Tip about names: avoid calling your own variables sum, mean, c or t, because those are the names of built-in functions. R can usually cope, but it confuses humans. Use names like total, avg or result.

5.5 Passing arguments: by position, by name, or by default

There are three ways to hand values to a function, and you will use all of them.

Code (type this)	What it does, in simple words	What you will see

`area_rect <- function(length = 5, width = 4) {`
`  area <- length * width`
`  return(area)`
`}`
	A function whose parameters have DEFAULT values: if you do not provide length it will be 5, and width will be 4.	*(nothing is shown)*

`area_rect(2, 3)`	By POSITION: the first value goes to the first parameter (length = 2), the second to the second (width = 3).	`[1] 6`
`area_rect(width = 8, length = 4)`	By NAME: you say which parameter each value is for, so the order does not matter. This is very clear to read.	`[1] 32`
`area_rect()`	Using the DEFAULTS: no arguments at all, so length is 5 and width is 4.	`[1] 20`
`area_rect(3)`	Mixed: the 3 fills the first parameter (length); width uses its default 4.	`[1] 12`
`area_rect(width = 10)`	Only change one value by name; length keeps its default 5.	`[1] 50`
`divide <- function(a, b) a / b` `divide(2, 10)`	Be careful with position! This is 2 divided by 10.	`[1] 0.2`
`divide(b = 2, a = 10)`	Using names makes your intention clear: 10 divided by 2.	`[1] 5`

Rule	What it means	What happens if you break it
Right number of arguments	Supply a value for every parameter without a default	Error: argument "b" is missing, with no default
No extras	Do not pass more arguments than the function has parameters	Error: unused argument
Defaults	A parameter with = value is optional	The default is used when it is left out

Rule	What it means	What happens if you break it
Return value	return() sends the answer back and ends the function immediately	Without it, R returns the value of the last line

5.6 Getting results back: return, lists and invisible results

Code (type this)	What it does, in simple words	What you will see
`rectangle_info <- function(length, width) {  area <- length * width  perimeter <- 2 * (length + width)  result <- list(Area = area, Perimeter = perimeter)  return(result) }`	A function can return only ONE object. To send back several results, put them in a list (a toolbox) with names, and return the list.	*(nothing is shown)*
`res <- rectangle_info(2, 3)`	Call it and save the list.	*(nothing is shown)*
`res$Area`	Open the Area compartment with the dollar sign.	`[1] 6`
`res$Perimeter`	And the Perimeter compartment.	`[1] 10`
`res["Area"]`	Single brackets give a smaller list that still carries the label.	`$Area [1] 6`
`double_it <- function(x) {  y <- x * 2 } double_it(4)`	TRAP: the last line is an assignment, so R returns the value invisibly and shows nothing when you call it.	*(nothing is shown)*
`print(double_it(4))`	The value is still there. Wrap the call in print() to see it, or end the function with the bare name y.	`[1] 8`
`z <- double_it(4) z`	Saved into z, the value is available as usual.	`[1] 8`

5.7 Built-in functions versus functions you write

R comes with thousands of ready-made built-in functions for maths, statistics, text and files. You write your own (user-defined) functions for jobs that are specific to your problem.

Category	Some built-in functions
Mathematical	abs(), sqrt(), round(), exp(), log(), sin(), cos(), tan()
Statistical	mean(), median(), var(), sd(), cor(), min(), max()
Data manipulation	unique(), subset(), aggregate(), order(), sort()
File input/output	read.csv(), write.csv(), read.table(), write.table()

Code (type this)	What it does, in simple words	What you will see
`sum(4:6)`	Built-in: add up 4, 5, 6.	`[1] 15`

Code (type this)	What it does, in simple words	What you will see
`max(4:6)`	Built-in: the biggest.	`[1] 6`
`min(4:6)`	Built-in: the smallest.	`[1] 4`
`nums <- c(10, 20, 30, 40)` `mean(nums)`	Built-in statistics: the average.	`[1] 25`
`sd(nums)`	The standard deviation: how spread out the numbers are.	`[1] 12.90994`

A first function that decides: even or odd?

Code (type this)	What it does, in simple words	What you will see

`even_odd <- function(x) {`
`  if (x %% 2 == 0) {`
`  return("Even")`
`  } else {`
`  return("Odd")`
`  }`
`}`
	A function with an if ... else inside. x %% 2 gives the remainder after dividing by 2. A remainder of 0 means even.	*(nothing is shown)*

`even_odd(4)`	4 divided by 2 leaves 0.	`[1] "Even"`
`even_odd(3)`	3 divided by 2 leaves 1.	`[1] "Odd"`

`divisible_by_5 <- function(n) {`
`  if (n %% 5 == 0) {`
`  return("number is divisible by 5")`
`  } else {`
`  return("number is not divisible by 5")`
`  }`
`}`
`divisible_by_5(100)`
	Another yes/no function.	

`[1] "number is divisible by 5"`		
`divisible_by_5(4)`	4 divided by 5 leaves 4.	
`[1] "number is not divisible by 5"`		

`divisible <- function(a, b) {`
`  if (a %% b == 0) {`
`  return(paste(a, "is divisible by", b))`
`  } else {`
`  return(paste(a, "is not divisible by", b))`
`  }`
`}`
`divisible(36, 6)`
	Two parameters this time, and paste() builds a friendly sentence.	

`[1] "36 is divisible by 6"`		
`divisible(7, 3)`	7 is not divisible by 3.	
`[1] "7 is not divisible by 3"`		

`divisible <- function(a, b = 3)`
`{`
`  if (a %% b == 0) {`
`  return(paste(a, "is divisible by", b))`
`  } else {`
` return(paste(a, "is not
	The same function with a DEFAULT: b = 3. If you give only a, it checks divisibility by 3.	

Code (type this)	What it does, in simple words	What you will see
`divisible by", b))   } } divisible(12)`		
`[1] "12 is divisible by 3"`		
`divisible(10, 5)`	Giving both arguments overrides the default.	
`[1] "10 is divisible by 5"`		

5.8 More about arguments: dots, functions as arguments, anonymous functions

The dots argument (...)

Three dots (...) mean "any number of extra arguments". It is useful when you do not know in advance how many values someone will pass.

Code (type this)	What it does, in simple words	What you will see
`total_of <- function(...) {   sum(...) } total_of(1, 2, 3, 4)`	The dots catch all the arguments and pass them along to sum().	`[1] 10`
`count_args <- function(...) {   length(list(...)) } count_args(5, "a", TRUE)`	list(...) gathers the extra arguments into a list, and length() counts them.	`[1] 3`
`describe <- function(n, ...) {   items <- list(n, ...)   paste(items, collapse = " ") } describe(5, 1L, 6i, TRUE, "hello", "dots")`	n takes the first argument; the dots take the rest. paste() with collapse glues everything into one sentence, even mixed types.	
`[1] "5 1 0+6i TRUE hello dots"`		

Passing a function as an argument

In R a function is just another object, so you can hand one function to another. This is the idea behind sapply() and across() that you met in the iteration chapter.

Code (type this)	What it does, in simple words	What you will see
`operate <- function(x, y, func) {   func(x, y) }`	operate() takes two numbers and a function, then applies that function to the numbers.	*(nothing is shown)*
`add <- function(a, b) a + b multiply <- function(a, b) a * b operate(3, 4, add)`	Pass add (no brackets!): 3 + 4.	`[1] 7`
`operate(3, 4, multiply)`	Pass multiply: 3 times 4.	`[1] 12`
`operate(3, 4, max)`	Built-in functions can be passed too: the larger of 3 and 4.	`[1] 4`
`apply_to <- function(x, fun2) {`	Another example: apply any summary	`[1] 2.5`

Code (type this)	What it does, in simple words	What you will see

`  return(fun2(x))`
`}`
`apply_to(c(1, 2, 3, 4), mean)`
	function to a vector.	

`(\(x) x + 1)(5)`	An anonymous function: a nameless, throw-away function written on the spot. \(x) x + 1 adds one, and the (5) at the end calls it straight away.	`[1] 6`

5.9 Lazy evaluation and missing arguments

R is lazy in a helpful way: it only looks at an argument at the moment the function actually needs it. So an argument you never use does not cause any problem, even if you do not supply it. An error appears only when the missing argument is used.

Code (type this)	What it does, in simple words	What you will see

`cylinder <- function(diameter, length, radius) {`
`  volume <- pi * diameter^2 * length / 4`
`  return(volume)`
`}`
`cylinder(5, 10)`
	radius is a parameter, but the body never uses it. We did not supply it, yet there is no error, because R never needed it.	`[1] 196.3495`

`cylinder2 <- function(diameter, length, radius) {`
`  volume <- pi * diameter^2 * length / 4`
`  print(radius)`
`  return(volume)`
`}`
`cylinder2(5, 10)`
	Now the body tries to print radius, which was never supplied. R must evaluate it, and fails. (Left out of the ready-made R file.)	

`Error in print(radius) :` `  argument "radius" is missing, with no default`		

`greet <- function(name) {`
`  if (missing(name)) {`
`  "Hello, stranger!"`
`  } else {`
`  paste("Hello,", name)`
`  }`
`}`
`greet()`
	missing(name) asks "did the caller leave this out?". This lets your function cope gracefully instead of crashing.	`[1] "Hello, stranger!"`

`greet("Amina")`	With an argument supplied, the else branch runs.	`[1] "Hello, Amina"`

5.10 Scope: where do variables live?

Variables created INSIDE a function are local: they exist only while the function runs, like notes written on a scrap of paper that is thrown away afterwards. Variables created OUTSIDE (in your Console or script) are global. A function can READ global variables if it cannot find one inside, but changes it makes to its own variables do not touch the global ones.

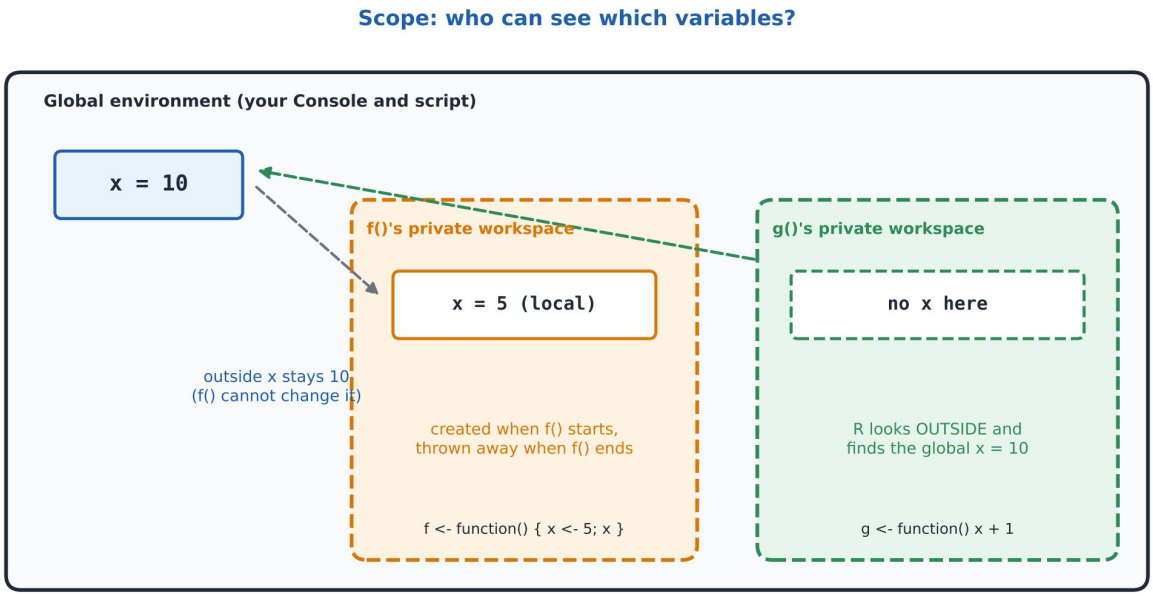


Figure 13: f() has its own private x that disappears when it ends. g() has no x of its own, so R looks outside.

Code (type this)	What it does, in simple words	What you will see

`x <- 10`
`f <- function() {`
`  x <- 5`
`  x`
`}`
`f()`
	Inside f(), a new local x is created with value 5, and that is what f() returns.	`[1] 5`

`x`	The global x is still 10. The function never changed it.	`[1] 10`
`g <- function() x + 1` `g()`	g() has no x of its own, so R looks outside and finds the global x (10).	`[1] 11`

`h <- function() {`
`  secret <- 99`
`}`
`h()`
`secret`
	secret only existed inside h(). Afterwards it is gone. (Left out of the ready-made R file.)	

`Error: object 'secret' not found`		

`counter <- 0`
`add_one <- function() {`
`  counter <- counter + 1`
`}`
`add_one()`
`counter`
	TRAP: add_one() changes only its own LOCAL copy of counter. The global counter is still 0.	`[1] 0`

`add_one <- function(x) x + 1`
`counter <- add_one(counter)`
`counter`
	FIX: pass the value in as an argument, return the new value, and save it yourself. This is the clean way to update a variable.	`[1] 1`

5.11 Control statements inside functions: the real power

Functions become really useful once they contain decisions and loops. The control statements from chapters 2 and 3 simply sit inside the curly brackets of a function.

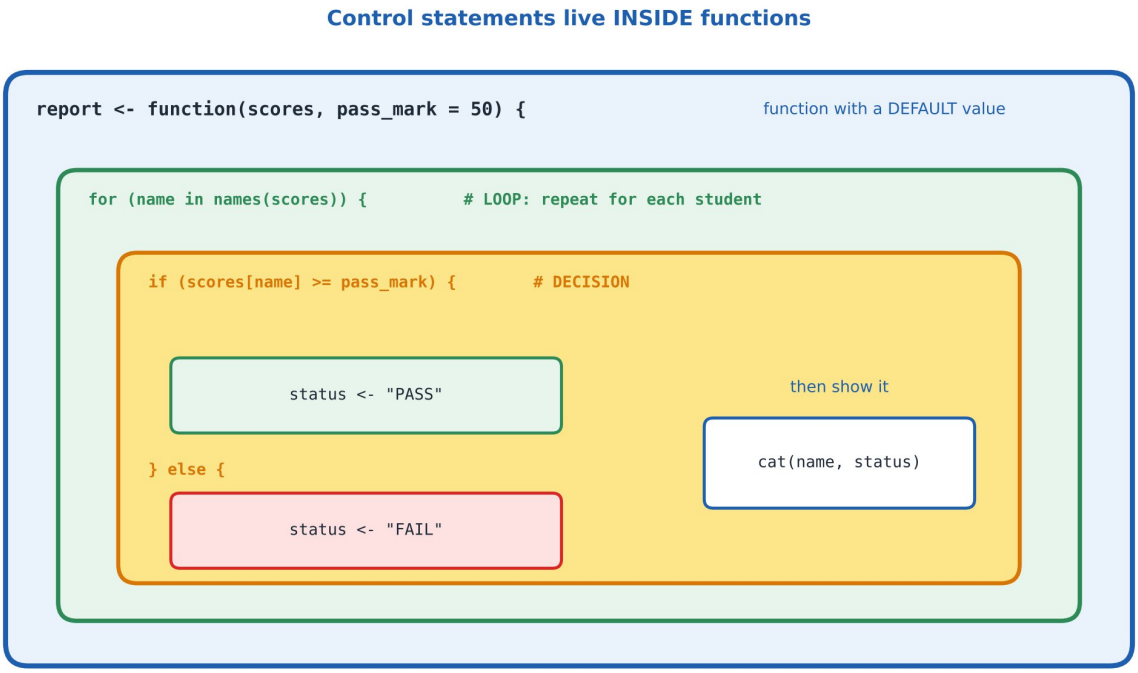


Figure 14: A function can contain a loop, and the loop can contain a decision. Notice the default value pass_mark = 50.

Code (type this)	What it does, in simple words	What you will see

`report <- function(scores, pass_mark = 50) {`
`  for (name in names(scores)) {`
`  if (scores[name] >= pass_mark) {`
`  status <- "PASS"`
`  } else {`
`  status <- "FAIL"`
`  }`
`  cat(name, ":", scores[name], "-", status, "\n")`
`  }`
`}`
	scores is a named vector. The function loops over each student's name, decides PASS or FAIL against pass_mark (default 50), and prints a line. The loop is inside the function, and the decision is inside the loop.	*(nothing is shown)*

`report(c(Ana = 72, Ben = 45, Chao = 50))`	Call with the default pass mark of 50. Chao scores exactly 50, which is >= 50, so he passes.	`Ana : 72 - PASS`
		`Ben : 45 - FAIL`
		`Chao : 50 - PASS`

`report(c(Ana = 72, Ben = 45), pass_mark = 70)`	Override the default: now the pass mark is 70.	`Ana : 72 - PASS` `Ben : 45 - FAIL`

`sum_to_n <- function(n) {`
`  total <- 0`
`  for (i in 1:n) {`
`  total <- total + i`
`  }`
`  total`
`}`
`sum_to_n(100)`
	A loop inside a function: adds 1 + 2 + ... + n. The famous answer for 100 is 5050.	`[1] 5050`

`first_multiple <- function(of, above) {`
`  n <- above + 1`
`  while (n %% of != 0) {`
`  n <- n + 1`
`  }`
	A while loop inside a function: the first multiple of 7 above 50.	`[1] 56`

Code (type this)	What it does, in simple words	What you will see

`  n`
`}`
`first_multiple(7, 50)`		

`check_login <-`
`function(attempts, correct =`
`"r2026") {`
`  tries <- 0`
`  for (guess in attempts) {`
`    tries <- tries + 1`
`    if (guess == correct) {`
`      return(paste("Welcome!`
`Attempt number", tries))`
`    }`
`    if (tries == 3) {`
`      return("Account locked")`
`    }`
`  }`
`  "Login failed"`
`}`
`check_login(c("abc", "r2026"))`
	A function with a loop, two decisions, a default value, and return() INSIDE the loop. return() ends the whole function immediately, so the loop stops too.	

`[1] "Welcome! Attempt number 2"`		
`check_login(c("a", "b", "c",` `"r2026"))`	Three wrong guesses lock the account before the correct one is tried.	`[1] "Account locked"`
`check_login("a")`	Only one wrong guess, then the guesses run out, so the last line runs.	`[1] "Login failed"`

5.12 Checking inputs: stop, warning, tryCatch

Good functions protect themselves. If someone gives a nonsense input, you can stop with a clear message, or warn and carry on.

Code (type this)	What it does, in simple words	What you will see

`safe_sqrt <- function(x) {`
`  if (!is.numeric(x)) {`
`    stop("x must be a number")`
`  }`
`  if (x < 0) {`
`    warning("negative input,`
`returning NA")`
`    return(NA)`
`  }`
`  sqrt(x)`
`}`
`safe_sqrt(16)`
	stop() ends the function with an error message. warning() shows a warning but lets the function carry on. For a valid input, the answer is returned normally.	`[1] 4`

`safe_sqrt(-4)`	A warning (negative input, returning NA) appears and the result is NA.	`[1] NA`
`safe_sqrt("a")`	The first check catches this. (Left out of the ready-made R file.)	
`Error in safe_sqrt("a") : x must be a number`		

`result <- tryCatch(log("a"),`
`error = function(e) "Oops, could`
`not take log")`
`result`
	tryCatch() lets you attempt something risky and say what to do if it fails, instead of stopping the whole script.	

`[1] "Oops, could not take log"`		

Code (type this)	What it does, in simple words	What you will see
`stopifnot(is.numeric(5), 5 > 0)`	stopifnot() checks that every statement is TRUE. If all are, nothing happens. If one is not, R stops with an error saying which.	*(nothing is shown)*

5.13 A function that calls itself: recursion (a peek)

Code (type this)	What it does, in simple words	What you will see

`factorial_r <- function(n) {`
`  if (n <= 1) {`
`  return(1)`
`  }`
`  n * factorial_r(n - 1)`
`}`
`factorial_r(5)`
	A function that uses itself. 5! = 5 x 4!, and 4! = 4 x 3!, and so on until n is 1. The if statement is the "stop sign" that ends the chain. Without it, the function would call itself forever.	`[1] 120`

`forever <- function(n) forever(n + 1)` `forever(1)`	A function with no stop sign. Luckily R protects you here: it stops by itself with an error instead of running forever. (Left out of the ready-made R file.)	
`Error: evaluation nested too deeply: infinite recursion / options(expressions=)?`		

6. Real-Life Problems Solved with Functions

Now we put everything together. Each problem below is a question you might meet in real life. We solve it with a function, explain what every line does, and test it. Notice how the functions combine decisions, loops and default values.

Problem 1: The electricity bill (decisions + defaults)

Question: Electricity costs 10 per unit for the first 50 units, 15 per unit for the next 50 units, and 20 per unit after that. Write a function that works out the bill for any number of units. Let the rates be changeable.

Code (type this)	What it does, in simple words	What you will see
`electricity_bill <- function(units, rate1 = 10, rate2 = 15, rate3 = 20) { if (units <= 50) { bill <- units * rate1 } else if (units <= 100) { bill <- 50 * rate1 + (units - 50) * rate2 } else { bill <- 50 * rate1 + 50 * rate2 + (units - 100) * rate3 } return(bill) }`	units is required. The three rates have default values, so the caller can change them if prices change. The ladder picks the right formula: up to 50 units, up to 100, or more. The first 50 units cost rate1 each, the next 50 cost rate2 each, and anything above 100 costs rate3 each.	*(nothing is shown)*
`electricity_bill(40)`	40 units, all at rate 10: 400.	`[1] 400`
`electricity_bill(80)`	First 50 at 10 = 500, the next 30 at 15 = 450. Total 950.	`[1] 950`
`electricity_bill(120)`	500 + 50 x 15 (750) + 20 x 20 (400) = 1650.	`[1] 1650`
`electricity_bill(80, rate2 = 12)`	Change one rate by name: the 30 middle units now cost 12 each (360), so 500 + 360.	`[1] 860`

Problem 2: Body mass index (decisions + returning a list)

Question: Write a function that takes a weight in kg and a height in metres and returns both the BMI and the health category. BMI is weight divided by height squared. Below 18.5 is Underweight, below 25 Normal, below 30 Overweight, otherwise Obese.

Code (type this)	What it does, in simple words	What you will see
`bmi_category <- function(weight_kg, height_m) { bmi <- weight_kg / height_m^2 if (bmi < 18.5) { category <- "Underweight" } else if (bmi < 25) { category <- "Normal" } else if (bmi < 30) { category <- "Overweight" } else { category <- "Obese"`	First calculate the BMI. The ladder picks a category. The last line returns TWO results together in a list: the BMI rounded to one decimal place, and the category.	*(nothing is shown)*

Code (type this)	What it does, in simple words	What you will see

`  }`
`  list(bmi = round(bmi, 1), category = category)`
`}`		

`result <- bmi_category(60, 1.70)` `result$bmi`	60 divided by 1.7 squared (2.89) is about 20.8.	`[1] 20.8`
`result$category`	20.8 is between 18.5 and 25.	`[1] "Normal"`
`bmi_category(95, 1.75)$category`	You can pick one item straight from the returned list with $. 95 / 3.0625 is about 31.	`[1] "Obese"`
`bmi_category(50, 1.80)$category`	50 / 3.24 is about 15.4.	`[1] "Underweight"`

Problem 3: Shop discount (defaults + checking input)

Question: A shop gives 10% off when the total of an order reaches 5000. Write a function final_price() that takes a price and a quantity. The discount and the threshold should be adjustable. Reject negative prices.

Code (type this)	What it does, in simple words	What you will see

`final_price <- function(price, quantity = 1, discount = 0.10, threshold = 5000) {`
`  if (price < 0 || quantity < 1) {`
`    stop("Price and quantity must be positive")`
`  }`
`  total <- price * quantity`
`  if (total >= threshold) {`
`    total <- total * (1 - discount)`
`  }`
`  total`
`}`
	Four parameters, three with defaults (quantity 1, discount 10%, threshold 5000). The first if protects against bad input using || (OR). Then the total is calculated, and the discount is applied only if the total reaches the threshold. The last line returns the total.	*(nothing is shown)*

`final_price(1200)`	One item at 1200: below the threshold, so no discount.	`[1] 1200`
`final_price(1200, 5)`	Five items make 6000, which is at least 5000, so 10% comes off: 5400.	`[1] 5400`
`final_price(1200, 5, discount = 0.2)`	A 20% promotion: 6000 x 0.8.	`[1] 4800`
`final_price(1200, quantity = 5, threshold = 7000)`	The threshold is raised to 7000, so 6000 gets no discount.	`[1] 6000`
`final_price(-5)`	A negative price triggers stop(). (Left out of the ready-made R file.)	
`Error in final_price(-5) : Price and quantity must be positive`		

Problem 4: Years to reach a savings goal (while loop + safety limit)

Question: You save 1000 and it grows by a fixed rate each year. How many years until you reach a target? The function should give up (return NA) after a maximum number of years, in case the target can never be reached.

Code (type this)	What it does, in simple words	What you will see

`years_to_target <- function(start, rate, target, max_years = 100) {`
`  balance <- start`
`  years <- 0`
`  while (balance < target && years < max_years) {`
`  balance <- balance * (1 + rate)`
`  years <- years + 1`
`  }`
`  if (balance < target) {`
`  return(NA)`
`  }`
`  years`
`}`
	The while condition has TWO parts joined by && : keep going while the target is not reached AND we have not hit the safety limit max_years. That safety limit makes an infinite loop impossible. After the loop, if the target was still not reached, return NA ("not available"); otherwise return the number of years.	*(nothing is shown)*

`years_to_target(1000, 0.10, 2000)`	At 10% a year, 1000 doubles in 8 years.	`[1] 8`
`years_to_target(1000, 0, 2000)`	At 0% the balance never grows. The loop runs until the safety limit (100 years) and the function returns NA instead of running forever.	`[1] NA`

Problem 5: Temperature converter (switch + stop)

Question: Write a function that converts a temperature from Celsius into Fahrenheit (F), back from Fahrenheit to Celsius (C), or into Kelvin (K), depending on a unit chosen by the caller.

Code (type this)	What it does, in simple words	What you will see

`convert_temp <- function(value, to = "F") {`
`  switch(to,`
`  F = value * 9 / 5 + 32,`
`  C = (value - 32) * 5 / 9,`
`  K = value + 273.15,`
`  stop("Unknown unit: use F, C or K")`
`  )`
`}`
	switch picks the formula by the value of to. The default is "F". The last, unnamed case is the default for anything else: it stops with a helpful message. Each formula takes a Celsius value for F and K, and a Fahrenheit value for C.	*(nothing is shown)*

`convert_temp(100)`	100 Celsius to Fahrenheit (the default).	`[1] 212`
`convert_temp(212, "C")`	212 Fahrenheit back to Celsius.	`[1] 100`
`convert_temp(0, "K")`	0 Celsius in Kelvin.	`[1] 273.15`
`convert_temp(5, "X")`	An unknown unit hits the default case. (Left out of the ready-made R file.)	
`Error in convert_temp(5, "X") : Unknown unit: use F, C or K`		

Problem 6: Is it a leap year? (combined conditions)

Question: A year is a leap year if it is divisible by 4, except for century years, which must be divisible by 400. Write a function that answers TRUE or FALSE, then check several years at once.

Code (type this)	What it does, in simple words	What you will see

`is_leap <- function(year) {`
`  (year %% 4 == 0 && year %%`
`100 != 0) || year %% 400 == 0`
`}`
	The whole rule in one line: divisible by 4 AND NOT by 100, OR divisible by 400. The comparison gives TRUE or FALSE and the last line is returned.	*(nothing is shown)*

`is_leap(2024)`	2024 is divisible by 4 but not by 100.	`[1] TRUE`
`is_leap(1900)`	1900 is divisible by 100 but not by 400.	`[1] FALSE`
`sapply(c(1900, 2000, 2024,` `2026), is_leap)`	Use sapply() to run the function on many years at once (iteration!).	
`[1] FALSE  TRUE  TRUE FALSE`		

Problem 7: FizzBuzz (loop + ladder + default)

Question: Print the numbers from 1 to n. But for multiples of 3 print Fizz, for multiples of 5 print Buzz, and for multiples of both print FizzBuzz. This classic puzzle is used in interviews.

Code (type this)	What it does, in simple words	What you will see

`fizzbuzz <- function(n = 15) {`
`  for (i in 1:n) {`
`  if (i %% 15 == 0) {`
`  cat("FizzBuzz\n")`
`  } else if (i %% 3 == 0) {`
`  cat("Fizz\n")`
`  } else if (i %% 5 == 0) {`
`  cat("Buzz\n")`
`  } else {`
`  cat(i, "\n", sep = "")`
`  }`
`  }`
`}`
	A loop from 1 to n, with a ladder inside. The strictest test (multiple of 15) comes FIRST, otherwise 15 would stop at the Fizz rule. The default n = 15 is used if you give no value. sep = "" removes the space cat() would add.	*(nothing is shown)*

`fizzbuzz(7)`	The first seven numbers.	`1`
		`2`
		`Fizz`
		`4`
		`Buzz`
		`Fizz`
		`7`

Problem 8: Class statistics (list result + default pass mark)

Question: Given a vector of exam marks, return the average, the highest mark and how many students passed (default pass mark 50).

Code (type this)	What it does, in simple words	What you will see

`class_stats <- function(marks,`
`pass_mark = 50) {`
`  list(`
	Three results in one list. sum(marks >= pass_mark) counts the TRUE answers,	*(nothing is shown)*

Code (type this)	What it does, in simple words	What you will see

`    mean = mean(marks),`
`    highest = max(marks),`
`    passed = sum(marks >= pass_mark)`
`  )`
`}`
	so it counts the students who passed.	

`stats <- class_stats(c(45, 72, 58, 90, 33))` `stats$mean`	The average of the five marks: 298 / 5.	`[1] 59.6`
`stats$highest`	The best mark.	`[1] 90`
`stats$passed`	72, 58 and 90 are at least 50.	`[1] 3`
`class_stats(c(45, 72, 58, 90, 33), pass_mark = 70)$passed`	Raise the pass mark to 70: only 72 and 90 pass.	`[1] 2`

Problem 9: Restock alerts (loop + decision + named vector)

Question: A shop keeps stock counts in a named vector. Print a warning for every item that has fewer than a minimum number left.

Code (type this)	What it does, in simple words	What you will see

`restock <- function(stock, minimum = 10) {`
`  for (item in names(stock)) {`
`    if (stock[item] < minimum) {`
`      cat(item, "needs restocking:", stock[item], "left\n")`
`    }`
`  }`
`}`
	Loop over the item names. If an item's count is below the minimum, print an alert. Items with enough stock print nothing.	*(nothing is shown)*

`restock(c(pens = 25, books = 4, bags = 9))`	Books (4) and bags (9) are below 10. Pens are fine.	
`books needs restocking: 4 left` `bags needs restocking: 9 left`		

Bonus: a prime-number checker (loop + early return)

Code (type this)	What it does, in simple words	What you will see

`is_prime <- function(n) {`
`  if (n < 2) return(FALSE)`
`  if (n < 4) return(TRUE)`
`  for (i in 2:floor(sqrt(n))) {`
`    if (n %% i == 0) return(FALSE)`
`  }`
`  TRUE`
`}`
	A prime has no divisors other than 1 and itself. The function returns early as soon as it finds a divisor. It only needs to test divisors up to the square root of n, which makes it fast. If the loop finishes with no divisor found, the last line returns TRUE.	*(nothing is shown)*

`is_prime(11)`	11 has no divisors from 2 to 3.	`[1] TRUE`
`is_prime(9)`	9 is divisible by 3.	`[1] FALSE`
`Filter(is_prime, 1:20)`	Filter() keeps only the items for which the function returns TRUE: the primes up to 20.	

Code (type this)	What it does, in simple words	What you will see
`[1]  2  3  5  7 11 13 17 19`		

7. Common Errors with Functions and Control Structures

Red output means an error. The line after each mistake shows the fix. (The lines that cause errors are left commented out in the ready-made R file.)

Code (type this)	What it does, in simple words	What you will see
`add_num(1)`	MISTAKE: too few arguments. add_num needs both a and b, and R needs b when it reaches a + b.	
`Error in add_num(1) : argument "b" is missing, with no default`		
`add_num(1, 2)`	FIX: supply every argument that has no default.	`[1] 3`
`add_num(1, 2, 3)`	MISTAKE: too many arguments. The function only has two parameters.	
`Error in add_num(1, 2, 3) : unused argument (3)`		
`divide(2, 10)`	MISTAKE: arguments in the wrong order. The answer is valid but not what you meant (10 divided by 2).	`[1] 0.2`
`divide(a = 10, b = 2)`	FIX: use names so the order cannot matter.	`[1] 5`

Code (type this)	What it does, in simple words	What you will see

`two_values <- function(a, b) {`
`  return(a, b)`
`}`
`two_values(1, 2)`
	MISTAKE: return() can send back only ONE object. (Left out of the ready-made R file.)	

`Error in return(a, b) : multi-argument returns are not permitted`		

`two_values <- function(a, b) {`
`  return(list(first = a, second = b))`
`}`
`two_values(1, 2)$second`
	FIX: bundle the results in a list.	`[1] 2`

`bad_sum <- function(v) {`
`  total <- 0`
`  for (x in v) {`
`  total <- total + x`
`  return(total)`
`  }`
`  total`
`}`
`bad_sum(c(1, 2, 3))`
	MISTAKE: return() is INSIDE the loop, so the function exits after the very first round and gives 1 instead of 6.	`[1] 1`

`good_sum <- function(v) {`
`  total <- 0`
`  for (x in v) {`
`  total <- total + x`
`  }`
`  total`
`}`
`good_sum(c(1, 2, 3))`
	FIX: finish the loop first, then return the result.	`[1] 6`

`uses_global <- function() w * 2` `uses_global()`	MISTAKE: the function relies on a variable w that does not exist. (Left out of the ready-made R file.)	
`Error in uses_global() : object 'w' not found`		

Code (type this)	What it does, in simple words	What you will see
`uses_arg <- function(w) w * 2` `uses_arg(21)`	FIX: give the function what it needs as an argument. Functions that depend only on their arguments are easy to trust and test.	`[1] 42`

Quick list of other traps

- **Forgetting the curly brackets** when an if, loop or function body has more than one line.
- **Writing } else { on separate lines** at the top level.
- **Using = when you mean ==** inside a condition.
- **Passing a function WITH brackets** (sapply(x, mean()) instead of sapply(x, mean)).
- **Expecting a function to change a global variable.** It changes its own local copy; return the new value instead.
- **Loops that cannot end.** Always make sure the condition can become FALSE, and add a safety limit when unsure.
- **Not testing.** Test every function on a normal input, an edge case (0, empty, negative) and a wrong input.

Good habits for functions and control flow

- Give functions short, clear verb-style names: area_of_circle, is_leap, final_price.
- Keep each function focused on one job. If it does three jobs, make three functions.
- Use default values for settings that rarely change (pass_mark = 50).
- Put comments at the top saying what the function expects and returns.
- Use stop() with a friendly message for bad inputs.
- Order else if ladders from the strictest condition to the loosest.
- Prefer vectorised code (ifelse, sum, x * 2) over loops for simple jobs.
- Test with small numbers before running big loops, and save your script first.

8. Practice Questions

Try every question yourself first. The practice file that comes with this document has space for your code. Answers follow.

Part A: Think about it (no code)

- A1. What is the difference between if ... else and ifelse()?
- A2. In an else if ladder, why must the strictest condition come first?
- A3. When would you choose a while loop instead of a for loop?
- A4. What does next do? What does break do?
- A5. Name three things an infinite loop can do to your computer, and two ways to stop one.
- A6. In function(length, width = 5), which are the parameters, and which one has a default?
- A7. In area_rect(4, width = 3), what are 4 and 3 called?
- A8. A variable is created inside a function. Can you use it after the function ends? Why?

Part B: Coding questions

- B1. Write is_even(n) that returns TRUE or FALSE. Test it with 10 and 7.
- B2. Write grade(marks) that returns A for 80 and above, B for 70 to 79, C for 60 to 69, and D otherwise. Test with 85, 72, 64, 40 using sapply().
- B3. Use a for loop to print "1 squared is 1", "2 squared is 4", up to 5.
- B4. Use a while loop to count how many times you can halve 100 before it drops below 1.
- B5. Write count_above(x, limit = 50) that counts the values of x above the limit. Test with c(30, 60, 80, 45), first with the default and then with limit = 40.
- B6. Write max_min(x) that returns the highest and the lowest value in a list. Test with c(5, 2, 9).
- B7. Write calc(a, b, op = "+") using switch for +, -, * and /. Test calc(8, 2), calc(8, 2, "/") and calc(8, 2, "*").
- B8. Use a loop and next to collect the numbers 1 to 10 that are NOT multiples of 3.
- B9. Use repeat and break to find the first number above 20 that is divisible by 6.
- B10. Use sapply() to find the sum of each item of list(a = 1:3, b = 4:6).

Part C: Real-life application questions

- C1. Taxi fare: the base fare is 100. The first 5 km cost 40 per km; every km after that costs 30 per km. Students get 10% off. Write taxi_fare(km, student = FALSE). Find the fare for 3 km, 8 km, and 8 km as a student.
- C2. Class summary: write summarise_marks(marks, pass_mark = 50) returning the mean, the highest mark and the number of passes. Use c(45, 72, 58, 90, 33).
- C3. Mobile data: the first 1 GB costs 100, the next 4 GB cost 80 per GB, and anything above 5 GB costs 60 per GB. Write data_cost(gb). Find the cost for 0.5, 3 and 8 GB.
- C4. Saving up: you save 1500 each month. Write months_to_save(goal, monthly, max_months = 1200) using a while loop. How many months to reach 10000? What if monthly is 0?

- C5. Password strength: write password_strength(pw). Strong if at least 8 characters and it contains a digit; Medium if at least 6 characters; otherwise Weak. Test "abc", "hello1", "secure2026".
- C6. Report a count: given scores <- c(Ana = 72, Ben = 45, Chao = 50, Dee = 88), use a function to count how many students scored at least 50 and name them.

Answers: Part A

- A1. if ... else decides ONE path for ONE TRUE/FALSE and chooses which code runs. ifelse() decides for every item of a vector and chooses values.
- A2. R stops at the first TRUE condition. A looser condition placed first would catch values meant for the stricter ones.
- A3. When you do not know in advance how many rounds are needed and you repeat until a condition changes (for example, until savings reach a goal).
- A4. next skips the rest of the current round and moves on; break stops the whole loop.
- A5. It can keep one CPU core at 100%, fill up memory (if the loop keeps growing something), freeze RStudio or crash R, and flood the Console. Stop it with Esc (or the red STOP button), or Session > Interrupt R / Terminate R.
- A6. length and width are the parameters; width has the default value 5.
- A7. They are arguments (the real values passed in).
- A8. No. It is local to the function and disappears when the function ends. To keep a result, return it.

Answers: Part B (code and output)

Code (type this)	What it does, in simple words	What you will see

`# B1`
`is_even <- function(n) {`
`  n %% 2 == 0`
`}`
`is_even(10)`
`is_even(7)`
	The remainder after dividing by 2 is 0 for even numbers. The comparison itself gives TRUE or FALSE.	`[1] TRUE`
		`[1] FALSE`

`# B2`
`grade <- function(marks) {`
`  if (marks >= 80) {`
`  "A"`
`  } else if (marks >= 70) {`
`  "B"`
`  } else if (marks >= 60) {`
`  "C"`
`  } else {`
`  "D"`
`  }`
`}`
`sapply(c(85, 72, 64, 40), grade)`
	A ladder inside a function. grade() works on a single mark, so sapply() applies it to each mark.	`[1] "A" "B" "C" "D"`

`# B3`
`for (i in 1:5) {`
`  cat(i, "squared is", i^2,`
`"\n")`
`}`
	i^2 squares the current number.	`1 squared is 1`
		`2 squared is 4`
		`3 squared is 9`
		`4 squared is 16`
		`5 squared is 25`

`# B4`
`n <- 100`
`halvings <- 0`
`while (n >= 1) {`
`  n <- n / 2`
`  halvings <- halvings + 1`
`}`
`halvings`
	100, 50, 25, 12.5, 6.25, 3.125, 1.5625, 0.78125: seven halvings until the value is below 1.	`[1] 7`

`# B5` `count_above <- function(x, limit`	x > limit gives TRUE/FALSE for each	`[1] 2` `[1] 3`

Code (type this)	What it does, in simple words	What you will see

`= 50) {`
`  sum(x > limit)`
`}`
`count_above(c(30, 60, 80, 45))`
`count_above(c(30, 60, 80, 45), limit = 40)`
	value, and sum() counts the TRUEs.	

`# B6`
`max_min <- function(x) {`
`  list(highest = max(x), lowest = min(x))`
`}`
`res <- max_min(c(5, 2, 9))`
`res$highest`
`res$lowest`
	Return a named list to send back two results.	`[1] 9`
		`[1] 2`

`# B7`
`calc <- function(a, b, op = "+")`
`{`
`  switch(op,`
`  "+" = a + b,`
`  "-" = a - b,`
`  "*" = a * b,`
`  "/" = a / b,`
`  stop("Unknown operator")`
`  )`
`}`
`calc(8, 2)`
`calc(8, 2, "/")`
`calc(8, 2, "*")`
	The operator text is matched by switch. The default value of op is "+".	`[1] 10`
		`[1] 4`
		`[1] 16`

`# B8`
`keep <- c()`
`for (i in 1:10) {`
`  if (i %% 3 == 0) {`
`  next`
`  }`
`  keep <- c(keep, i)`
`}`
`keep`
	next skips multiples of 3, so they are never added.	

`[1]  1  2  4  5  7  8 10`		

`# B9`
`n <- 21`
`repeat {`
`  if (n %% 6 == 0) {`
`  break`
`  }`
`  n <- n + 1`
`}`
`n`
	Start at 21 and go up one at a time until a multiple of 6 is found.	`[1] 24`

`# B10` `sapply(list(a = 1:3, b = 4:6), sum)`	sum is applied to each list item.	`a  b` `6 15`

Answers: Part C (code and output)

Code (type this)	What it does, in simple words	What you will see

`# C1`
`taxi_fare <- function(km, student = FALSE) {`
	Base fare plus per-km charges using a decision, then a second decision applies the student discount. student	`[1] 220`
		`[1] 390`
		`[1] 351`

Code (type this)	What it does, in simple words	What you will see

`  if (km <= 5) {`
`    fare <- 100 + km * 40`
`  } else {`
`    fare <- 100 + 5 * 40 + (km - 5) * 30`
`  }`
`  if (student) {`
`    fare <- fare * 0.9`
`  }`
`  fare`
`}`
`taxi_fare(3)`
`taxi_fare(8)`
`taxi_fare(8, student = TRUE)`
	= FALSE is a default, so ordinary riders do not need to mention it. 3 km: 100 + 120. 8 km: 100 + 200 + 90. Student: 390 x 0.9.	

`# C2`
`summarise_marks <- function(marks, pass_mark = 50) {`
`  list(mean = mean(marks), highest = max(marks), passed = sum(marks >= pass_mark))`
`}`
`s <- summarise_marks(c(45, 72, 58, 90, 33))`
`s$mean`
`s$highest`
`s$passed`
	A list returns three results. The mean is 298 / 5 = 59.6.	`[1] 59.6`
		`[1] 90`
		`[1] 3`

`# C3`
`data_cost <- function(gb) {`
`  if (gb <= 1) {`
`    100`
`  } else if (gb <= 5) {`
`    100 + (gb - 1) * 80`
`  } else {`
`    100 + 4 * 80 + (gb - 5) * 60`
`  }`
`}`
`data_cost(0.5)`
`data_cost(3)`
`data_cost(8)`
	A ladder with three price bands. 3 GB: 100 + 2 x 80. 8 GB: 100 + 320 + 3 x 60.	`[1] 100`
		`[1] 260`
		`[1] 600`

`# C4`
`months_to_save <- function(goal, monthly, max_months = 1200) {`
`  saved <- 0`
`  months <- 0`
`  while (saved < goal && months < max_months) {`
`    saved <- saved + monthly`
`    months <- months + 1`
`  }`
`  if (saved < goal) {`
`    return(NA)`
`  }`
`  months`
`}`
`months_to_save(10000, 1500)`
`months_to_save(10000, 0)`
	The safety limit max_months makes sure the loop cannot run forever when monthly is 0. Seven months give 10500, which is at least 10000.	`[1] 7`
		`[1] NA`

`# C5`
`password_strength <- function(pw) {`
`  n <- nchar(pw)`
`  has_digit <- grepl("[0-9]",`
	nchar() counts characters; grepl("[0-9]", pw) asks whether the text contains any digit.	`[1] "Weak"`
		`[1] "Medium"`
		`[1] "Strong"`

Code (type this)	What it does, in simple words	What you will see

pw)
  if (n >= 8 && has_digit) {
  "Strong"
  } else if (n >= 6) {
  "Medium"
  } else {
  "Weak"
  }
}
password_strength("abc")
password_strength("hello1")
password_strength("secure2026")		

# C6 	
pass_list <- function(scores, pass_mark = 50) {
  passed <- names(scores)[scores >= pass_mark]
  cat("Passed:", length(passed), "\n")
  cat("Names:", passed, "\n")
}
pass_list(c(Ana = 72, Ben = 45, Chao = 50, Dee = 88))
	scores >= pass_mark is a TRUE/FALSE vector. Using it inside names(scores)[...] picks the names of the students who passed.	Passed: 3
		Names: Ana Chao Dee

9. Free Resources to Keep Learning

Free books and chapters

- **R for Data Science (2nd edition):** https://r4ds.hadley.nz The full free book. See its chapters on Functions and Iteration (the iteration chapter is the source of the across(), map() and walk() ideas here).
- **R4DS: Functions:** https://r4ds.hadley.nz/functions Beginner-friendly guide to writing your own functions.
- **R4DS: Iteration:** https://r4ds.hadley.nz/iteration across(), map(), reading many files and saving many outputs.
- **Hands-On Programming with R:** https://rstudio-education.github.io/hopr/ Written for non-programmers; includes chapters on programs and loops.
- **Advanced R: Functions:** https://adv-r.hadley.nz/functions.html For later: a deeper look at how functions work.
- **Advanced R: Functionals:** https://adv-r.hadley.nz/functionals.html For later: map(), apply and friends in depth.
- **An Introduction to R (official manual):** https://cran.r-project.org/doc/manuals/r-release/R-intro.html See the chapter on grouping, loops and conditional execution.

Package websites

- **purrr:** https://purrr.tidyverse.org/ Documentation for map(), walk(), possibly() and more.
- **dplyr:** https://dplyr.tidyverse.org/ Documentation for across(), mutate() and summarize().
- **Tidyverse style guide:** https://style.tidyverse.org/ How to write clean, readable R code.

Practice and community

- **swirl:** https://swirlstats.com/ Learn R inside R with interactive lessons.
- **Posit Cloud:** https://posit.cloud/ R in your browser, no installation.
- **Posit Education: Beginners:** https://education.rstudio.com/learn/beginner/ A curated path for new R learners.
- **Posit Community forum:** https://forum.posit.co/ A friendly place to ask questions.

> **About these links:** The examples here were adapted from common R tutorials (including GeeksforGeeks articles on functions, arguments and decision making, and the Iteration chapter of R for Data Science), with corrections and extra explanation. The book, Posit Education and tibble links were checked earlier; the chapter and package pages are standard addresses that were not individually opened, so if one does not load, search for the title.

Where to go next

You can now make R decide, repeat and reuse. Next steps: reading real data from CSV files, drawing charts with ggplot2, and learning the tidyverse tools dplyr and purrr in more depth. Keep practising a little each day, and always test small before running big. You are doing wonderfully.